

Question 1

A job allows one concurrent run and has queueing enabled. A scheduled run is active when an operator starts another run manually, and no workspace-level limit has been reached.

What happens to the manual run?

- A. It waits in the queue and starts once the active run has finished, provided that it stays within the queue's own time limit
- B. It cancels the currently active run and starts right away, with the manual request taking priority over the scheduled run
- C. It starts immediately, since the maximum concurrent runs setting counts only the scheduled runs toward the job's run limit
- D. It is skipped, since queueing holds runs back when a workspace-level concurrency limit is reached, not the job's own run limit

Correct answer: A

- A. It waits in the queue and starts once the active run has finished, provided that it stays within the queue's own time limit

Explanation

Correct answer:

It waits in the queue and starts once the active run has finished, provided that it stays within the queue's own time limit

Why this is correct:

With queueing enabled, reaching the job's maximum concurrent runs places the new run in the queue instead of skipping it. Once the active run ends, the queued manual run can start as long as it remains within the queue's 48-hour limit.

Why the other options are wrong:

It starts immediately, since the maximum concurrent runs setting counts only the scheduled runs toward the job's run limit: The concurrency setting applies to all runs of the job, including manually started runs. A manual request does not bypass the configured limit.

It cancels the currently active run and starts right away, with the manual request taking priority over the scheduled run: Queueing does not prioritize manual runs by canceling active work. It waits for capacity rather than interrupting the scheduled execution.

It is skipped, since queueing holds runs back when a workspace-level concurrency limit is reached, not the job's own run limit: The job's own maximum concurrent runs is explicitly one of the limits that can cause a run to queue. Queueing is not restricted to workspace-wide capacity limits.

References:

1. **Configure and edit Lakeflow Jobs:** <https://docs.databricks.com/aws/en/jobs/configure-job>

Question 2

An append-only source table has a unique `event_id` that only increases, so no lower ID ever appears later. A batch pipeline stores the highest loaded ID as `last_id`, and each run must read only new events.

```
SELECT * FROM src.events WHERE _____
```

Which predicate and process meet this requirement?

- A. `event_id > last_id`, with `last_id` advanced only after the load into the target table has committed successfully
- B. `event_id < last_id`, with `last_id` advanced after each load, which rereads rows that earlier runs already loaded
- C. `event_id > last_id`, with `last_id` advanced before the load runs, so a failed load still moves the watermark forward
- D. `event_id >= last_id`, with `last_id` advanced after each load, which reloads the last loaded row on every run

Correct answer: A

A. `event_id > last_id`, with `last_id` advanced only after the load into the target table has committed successfully

Explanation

Correct answer:

`event_id > last_id`, with `last_id` advanced only after the load into the target table has committed successfully

Why this is correct:

A strict `>` watermark selects only IDs above the last successfully committed high-water mark, preventing the boundary row from being loaded twice. Advance `last_id` only after the destination transaction commits; otherwise a failed write could move the checkpoint past events that never landed.

Why the other options are wrong:

`event_id >= last_id`, with `last_id` advanced after each load, which reloads the last loaded row on every run: Using `>=` repeats the row whose ID equals the saved watermark on each run. Strict greater-than avoids rereading the last successful ID.

`event_id > last_id`, with `last_id` advanced before the load runs, so a failed load still moves the watermark forward: Advancing before commit creates a gap risk: if the target load fails, the next run starts past IDs that were never written. Checkpoints must reflect successful durable progress.

event_id < last_id, with last_id advanced after each load, which rereads rows that earlier runs already loaded: Less-than selects historical IDs below the watermark rather than newly appended higher values. It would repeatedly reread old events and miss later ones.

References:

1. Query-based connector reference:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/query-based-reference>

Question 3

A colleague merged changes into main on the remote repository. An engineer's Git folder is on main with no local edits, and she wants those merged commits in her folder.

Which Git operation should she run?

- A. Pull in the Git folder, which fetches the new commits from the remote and merges them into her local copy of the current branch
- B. Create a new branch from main in the Git folder, which copies the latest remote commits over into that new working branch
- C. Push from the Git folder, which synchronizes the remote's new commits into her Git folder as well as sending her own local changes upstream
- D. Commit in the Git folder, which records the merged remote work locally before any fetch or merge with the remote can actually take place

Correct answer: A

A. Pull in the Git folder, which fetches the new commits from the remote and merges them into her local copy of the current branch

Explanation

Correct answer:

Pull in the Git folder, which fetches the new commits from the remote and merges them into her local copy of the current branch

Why this is correct:

The Git folder is already on the branch that received the remote merge and has no local changes. Pull fetches the branch's new remote commits and integrates them into the checked-out local branch, which is the synchronization operation needed here.

Why the other options are wrong:

Create a new branch from main in the Git folder, which copies the latest remote commits over into that new working branch: Creating another branch changes the local branch pointer but does not fetch new commits by itself. A new branch from a stale local main would still lack the merged remote work.

Commit in the Git folder, which records the merged remote work locally before any fetch or merge with the remote can actually take place: Commit records local modifications as a new commit. It does not retrieve the commits that the colleague pushed to the remote.

Push from the Git folder, which synchronizes the remote's new commits into her Git folder as well as sending her own local changes upstream: To update this folder, the operation must fetch remote history and integrate it into the checked-out branch; uploading local commits would leave the folder

unchanged.

References:

1. **Git folders:** <https://docs.databricks.com/aws/en/repos>

Question 4

After a deployment bumps one Python package, a notebook fails at import. The error names an installed dependency whose version falls outside the range the new package requires. Installation succeeded, and no Spark job has started.

What should the engineer investigate first?

- A. Restarting the Python process with `dbutils.library.restartPython()`, which reloads the same installed library versions and then clears the error
- B. Check the currently installed package versions against their declared dependency constraints, then pin a version of the conflicting library that fits
- C. Moving the package from a notebook-scoped to a cluster-scoped library instead, which resolves version ranges differently when the cluster starts
- D. Clearing out the notebook's cached outputs and rerunning every cell, since stale state left from the old package version triggers the error

Correct answer: B

B. Check the currently installed package versions against their declared dependency constraints, then pin a version of the conflicting library that fits

Explanation

Correct answer:

Check the currently installed package versions against their declared dependency constraints, then pin a version of the conflicting library that fits

Why this is correct:

The error occurs during Python import, before Spark starts, and identifies a dependency outside the newly installed package's declared version range. Compare installed versions with dependency constraints, then pin a compatible version so the environment's packages can be resolved together.

Why the other options are wrong:

Moving the package from a notebook-scoped to a cluster-scoped library instead, which resolves version ranges differently when the cluster starts: Changing library scope alters where or when a package is installed, not the compatibility rules between its dependencies. The same incompatible versions can still produce the import error.

Clearing out the notebook's cached outputs and rerunning every cell, since stale state left from the old package version triggers the error: Cached notebook output does not affect the interpreter's installed package graph. Clearing outputs cannot repair an incompatible dependency version.

Restarting the Python process with `dbutils.library.restartPython()`, which reloads the same

installed library versions and then clears the error: Restarting Python reloads the existing environment; it does not select new package versions. A restart helps after a successful library change, not instead of resolving this conflict.

References:

1. Install libraries from a package repository:

<https://docs.databricks.com/aws/en/libraries/package-repositories>

Question 5

The DataFrame sizes has 4 distinct rows. The DataFrame services has 3 rows, two of which are identical. An engineer runs `sizes.crossJoin(services).distinct()` and counts the result.

How many rows does the result contain?

- A. 4, because `distinct()` keeps just one row per size once the same service values repeat across the combined output rows of the join
- B. 12, because a cross join multiplies the two input row counts and `distinct()` then cannot reduce the output of a cross join of distinct inputs
- C. 7, because a cross join simply appends one DataFrame's rows to the other's, and `distinct()` then finds no duplicate rows left to remove
- D. 8, because the repeated service row produces duplicate pairings with each size value that `distinct()` then collapses back into single rows

Correct answer: D

D. 8, because the repeated service row produces duplicate pairings with each size value that `distinct()` then collapses back into single rows

Explanation

Correct answer:

8, because the repeated service row produces duplicate pairings with each size value that `distinct()` then collapses back into single rows

Why this is correct:

A cross join pairs every distinct sizes row with each physical row of services, producing 12 rows before `distinct()`. Because two service rows are identical, each of four sizes appears twice with that same service value; deduplication collapses those four repeated pairs, leaving eight unique combinations.

Why the other options are wrong:

7, because a cross join simply appends one DataFrame's rows to the other's, and `distinct()` then finds no duplicate rows left to remove: A cross join forms a Cartesian product, not a row append. With four rows and three rows it produces 12 pairings before duplicates are removed.

4, because `distinct()` keeps just one row per size once the same service values repeat across the combined output rows of the join: Each of four distinct size values can pair with each of two distinct service values. Repetition in services removes one duplicate per size, not all but one service pairing.

12, because a cross join multiplies the two input row counts and `distinct()` then cannot reduce the output of a cross join of distinct inputs: The inputs are not both distinct: services contains a duplicate row. `distinct()` removes the duplicated size-service pairs, reducing 12 to 8.

References:

1. pyspark.sql.DataFrame:

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html>

Question 6

A scheduled Python notebook must reach a privately routed database from the company's own cloud network. The approved network path exists only for classic compute, and serverless connectivity has not been approved.

Which compute choice meets these constraints?

- A.** A job cluster running in the customer's own cloud account, which the approved route already reaches and which ends after every run finishes
- B.** An instance pool without an attached cluster, which then runs the scheduled notebook directly on its own idle instances each night
- C.** Serverless compute for jobs, whose network connectivity configuration can be approved at a later point without changing the job
- D.** A serverless SQL warehouse, on which a scheduled Python notebook task can run without needing interactive cluster access

Correct answer: A

A. A job cluster running in the customer's own cloud account, which the approved route already reaches and which ends after every run finishes

Explanation

Correct answer:

A job cluster running in the customer's own cloud account, which the approved route already reaches and which ends after every run finishes

Why this is correct:

Classic job compute runs in the customer's cloud account, so it can use the private route already approved there. A job cluster is created for the run and terminates afterward, fitting the network boundary and scheduled batch lifecycle.

Why the other options are wrong:

An instance pool without an attached cluster, which then runs the scheduled notebook directly on its own idle instances each night: An instance pool is a source of ready instances for clusters, not compute that runs a notebook on its own. The task still needs a cluster configured to use the pool.

A serverless SQL warehouse, on which a scheduled Python notebook task can run without needing interactive cluster access: A SQL warehouse is designed for SQL workloads, not a scheduled Python notebook task. Its network placement also does not meet the explicit approval limited to classic compute.

Serverless compute for jobs, whose network connectivity configuration can be approved at a

later point without changing the job: The team has not approved serverless networking, so choosing serverless compute assumes a connectivity capability that is not available for this workload.

References:

1. **Configure compute for jobs:** <https://docs.databricks.com/aws/en/jobs/compute>

Question 7

An engineer investigating an access complaint needs to list only the privileges that the analysts group holds on `analytics.gold.daily_revenue`, without changing any grants.

Which command should the engineer run?

- A. `DESCRIBE HISTORY analytics.gold.daily_revenue;`
- B. `SHOW GRANTS ON TABLE analytics.gold.daily_revenue;`
- C. `SHOW GRANTS analysts ON TABLE analytics.gold.daily_revenue;`
- D. `SHOW GRANTS analysts ON SCHEMA analytics.gold;`

Correct answer: C

C. `SHOW GRANTS analysts ON TABLE analytics.gold.daily_revenue;`

Explanation

Correct answer:

`SHOW GRANTS analysts ON TABLE analytics.gold.daily_revenue;`

Why this is correct:

`SHOW GRANTS <principal> ON TABLE ...` filters the inspection to grants associated with the named principal on that securable. It is read-only, so it answers the targeted audit question without changing permissions.

Why the other options are wrong:

`SHOW GRANTS analysts ON SCHEMA analytics.gold;`: The schema-level command inspects a different securable and would not isolate the group's grants on `daily_revenue`. The question asks about a specific table.

`DESCRIBE HISTORY analytics.gold.daily_revenue;`: `DESCRIBE HISTORY` reports table operations and versions, not access-control grants. It is useful for auditing data changes.

`SHOW GRANTS ON TABLE analytics.gold.daily_revenue;`: Without a principal, the command reports grants for all principals on the table. Supplying `analysts` narrows the result to the group under investigation.

References:

1. Manage privileges in Unity Catalog:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/?language=SQL>

Question 8

A Lakeflow Connect SQL Server pipeline overwrites destination rows whenever source rows change. Auditors now need every earlier version of each customer row, along with the time span during which each version was current.

Which change meets this need?

A. Read older table versions with Delta time travel, which keeps the past versions for as long as the auditors continue to need them

B. Run a full table refresh every night, which then rebuilds each destination table with the complete history of changes made in the source system

C. Keep SCD type 1 as is and enable change data feed on the destination tables, which keeps each overwritten row version permanently for later audits

D. Turn on history tracking (SCD type 2), which keeps the earlier row versions and also adds __START_AT and __END_AT columns to every row

Correct answer: D

D. Turn on history tracking (SCD type 2), which keeps the earlier row versions and also adds __START_AT and __END_AT columns to every row

Explanation

Correct answer:

Turn on history tracking (SCD type 2), which keeps the earlier row versions and also adds __START_AT and __END_AT columns to every row

Why this is correct:

SCD type 2 retains each changed row version and records when that version became active and when it ended using __START_AT and __END_AT. This is the appropriate history model when auditors need both prior values and their validity intervals; type 1 intentionally replaces old values.

Why the other options are wrong:

Run a full table refresh every night, which then rebuilds each destination table with the complete history of changes made in the source system: Rebuilding a snapshot recovers the source's present state, not the sequence of prior states and their effective dates. Frequent full refreshes can also erase maintained SCD history.

Keep SCD type 1 as is and enable change data feed on the destination tables, which keeps each overwritten row version permanently for later audits: Change data feed records table changes for downstream incremental consumers, but it does not change type 1's current-row storage into an indefinitely maintained SCD2 history table with validity columns.

Read older table versions with Delta time travel, which keeps the past versions for as long as the auditors continue to need them: Table-version snapshots have retention limits and describe the whole table at a point in time; they do not provide per-customer validity intervals for long-term auditing.

References:

1. Enable history tracking (SCD type 2):

<https://docs.databricks.com/gcp/en/ingestion/lakeflow-connect/scd>

Question 9

subscribers_df and **invoices_df** share a non-null **subscriber_id** column. The engineer needs every subscriber that has no matching invoice, with only the subscriber columns in the result and no duplicates introduced.

Which expression returns those rows?

- A. `subscribers_df.join(invoices_df, "subscriber_id", "left")`
- B. `subscribers_df.join(invoices_df, "subscriber_id", "left_semi")`
- C. `subscribers_df.join(invoices_df, "subscriber_id", "left").filter("subscriber_id IS NULL")`
- D. `subscribers_df.join(invoices_df, "subscriber_id", "left_anti")`

Correct answer: D

D. `subscribers_df.join(invoices_df, "subscriber_id", "left_anti")`

Explanation

Correct answer:

`subscribers_df.join(invoices_df, "subscriber_id", "left_anti")`

Why this is correct:

A left anti join returns rows from the left DataFrame that have no matching key in the right DataFrame. It emits only left-side columns, so it directly returns unmatched subscribers without invoice columns or row multiplication from multiple matching invoices.

Why the other options are wrong:

`subscribers_df.join(invoices_df, "subscriber_id", "left")`: Matching invoices remain in an outer-join result, so subscribers with a match are included too and repeated invoices can multiply their rows. The query needs only the no-match subset.

`subscribers_df.join(invoices_df, "subscriber_id", "left").filter("subscriber_id IS NULL")`: The shared join key is non-null, and an unmatched left join row still keeps its `subscriber_id`. Filtering that key for NULL therefore does not identify the unmatched rows.

`subscribers_df.join(invoices_df, "subscriber_id", "left_semi")`: The semi-join existence test selects subscribers that have an invoice, which is the complement of the unmatched population requested.

References:

1. JOIN: <https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-qry-select-join>

Question 10

A batch job copied a config that sets `spark.sql.files.maxPartitionBytes` to 512 MB. Its scan of eight 1 GB Parquet files runs only 16 tasks, leaving most of 64 cores idle; later stages are balanced.

Which change should be tested first, then remeasured?

- A. Raise the `spark.sql.shuffle.partitions` setting to 64, expecting the scan stage to split its input across as many tasks as there are cores
- B. Call `repartition(64)` right after the read, expecting the scan itself to run as 64 separate tasks rather than the current 16 tasks
- C. Lower the `spark.sql.files.maxPartitionBytes` setting, so the scan cuts each file into more input splits, then recheck the scan's task count
- D. Enable AQE partition coalescing, which lets the scan combine its input splits into the right number of tasks for the available cores

Correct answer: C

C. Lower the `spark.sql.files.maxPartitionBytes` setting, so the scan cuts each file into more input splits, then recheck the scan's task count

Explanation

Correct answer:

Lower the `spark.sql.files.maxPartitionBytes` setting, so the scan cuts each file into more input splits, then recheck the scan's task count

Why this is correct:

`spark.sql.files.maxPartitionBytes` influences how file bytes are grouped into scan partitions. Lowering the 512 MB target should create more input splits from the eight large splittable Parquet files, increasing scan task parallelism; then measure the scan stage to confirm the change helps.

Why the other options are wrong:

Raise the `spark.sql.shuffle.partitions` setting to 64, expecting the scan stage to split its input across as many tasks as there are cores: Shuffle partition settings govern post-shuffle stages, not the file scan. Raising that value will not turn the observed 16 input tasks into 64 scan tasks.

Call `repartition(64)` right after the read, expecting the scan itself to run as 64 separate tasks rather than the current 16 tasks: `repartition` is a later transformation; Spark must first finish reading the source before it can reshuffle rows. It cannot increase parallelism in the scan that precedes it.

Enable AQE partition coalescing, which lets the scan combine its input splits into the right number of tasks for the available cores: AQE coalescing optimizes shuffle partitions, not file input

splits. It is not the control for this under-parallelized scan stage.

References:

1. **Set Spark configuration properties on Databricks:** <https://docs.databricks.com/aws/en/spark/conf>

Question 11

Ownership of sales.gold.revenue must move from the analytics group to the platform_admins group. Nobody holds the **MANAGE** privilege on the table or on its parent schema and catalog.

Which **TWO** principals can perform the transfer?

- A. A user who holds **BROWSE** on the catalog that contains the revenue table, which exposes its metadata to that particular user
- B. A user who holds both **SELECT** and **MODIFY** on the revenue table itself, which allows reading and changing its stored data rows
- C. A metastore admin assigned to the metastore that contains the revenue table, which holds authority over all of its securables
- D. A user in both analytics and platform_admins, who acts on behalf of the analytics group as the table's current owner
- E. A user who holds **USE SCHEMA** and **CREATE TABLE** on the parent schema, which lets that user create new tables right beside it

Correct answers: C, D

- C. A metastore admin assigned to the metastore that contains the revenue table, which holds authority over all of its securables
- D. A user in both analytics and platform_admins, who acts on behalf of the analytics group as the table's current owner

Explanation

Correct answer:

A metastore admin assigned to the metastore that contains the revenue table, which holds authority over all of its securables

A user in both analytics and platform_admins, who acts on behalf of the analytics group as the table's current owner

Why this is correct:

A metastore admin can transfer ownership of a Unity Catalog table. The current owner can also transfer it to a group that the acting user belongs to; the user belongs to platform_admins, so acting on behalf of analytics satisfies the second route described.

Why the other options are wrong:

A user who holds BROWSE on the catalog that contains the revenue table, which exposes its metadata to that particular user: BROWSE permits discovery and metadata visibility, not ownership changes. Ownership transfer requires current ownership, metastore-admin authority, container

ownership, or MANAGE.

A user who holds USE SCHEMA and CREATE TABLE on the parent schema, which lets that user create new tables right beside it: Schema usage and table creation let someone work within the schema but do not administer an existing table's ownership.

A user who holds both SELECT and MODIFY on the revenue table itself, which allows reading and changing its stored data rows: Row-level rights address access to records, not control of the table object. Changing ownership requires administrative authority on the securable, such as being its owner or holding MANAGE.

References:

1. Manage privileges in Unity Catalog:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/ownership>

Question 12

An Auto Loader stream uses an explicitly supplied schema with `failOnNewColumns`. It stopped because files added an approved business field that must become a regular column. Strict validation and existing progress must both be kept.

What should the engineer do before restarting?

- A. Switch the schema evolution mode to `rescue` and keep the supplied schema, so the approved field lands in the rescued data column as plain text
- B. Keep the supplied schema but delete the checkpoint directory, so the stream starts over from scratch and picks up the approved field automatically
- C. Switch the schema evolution mode to `addNewColumns` and drop the supplied schema, so later fields are then added without any validation checks
- D. Add the approved field to the supplied schema, keep the current schema evolution mode and the checkpoint location, then restart the stream

Correct answer: D

D. Add the approved field to the supplied schema, keep the current schema evolution mode and the checkpoint location, then restart the stream

Explanation

Correct answer:

Add the approved field to the supplied schema, keep the current schema evolution mode and the checkpoint location, then restart the stream

Why this is correct:

With `failOnNewColumns`, an unexpected field stops the stream without automatically widening the supplied schema. Add the approved field to that schema, keep the strict mode and existing checkpoint, then restart; this preserves stream progress and rejects any other unapproved additions.

Why the other options are wrong:

Keep the supplied schema but delete the checkpoint directory, so the stream starts over from scratch and picks up the approved field automatically: Deleting the checkpoint discards recorded progress and risks reprocessing data from the beginning. It also does not add the approved field to the explicitly supplied schema.

Switch the schema evolution mode to `addNewColumns` and drop the supplied schema, so later fields are then added without any validation checks: `addNewColumns` relaxes validation by accepting future fields automatically, which conflicts with the request to preserve strict approval. Dropping the schema also removes explicit schema control.

Switch the schema evolution mode to rescue and keep the supplied schema, so the approved field lands in the rescued data column as plain text: Rescue mode would store the field outside the regular typed schema, so downstream consumers would not receive it as an approved business column. The field needs to be added to the supplied schema.

References:

1. Schema evolution in Databricks:

<https://docs.databricks.com/aws/en/data-engineering/schema-evolution>

Question 13

A nightly job usually finishes in 30 minutes. The team wants an email whenever a run is still going after 45 minutes, while letting that run continue to completion.

Which configuration fits?

- A. A failure notification set on the job, which also fires for any run that exceeds its usual run duration by more than half an hour
- B. A 45-minute duration warning threshold with notifications enabled for it, which alerts the team but leaves the run going
- C. A 45-minute timeout set on the job with one retry, which emails the team at the time limit and then restarts the run so it can finish
- D. A 45-minute timeout set on the job, which emails the team when the time limit is reached and then lets the run carry on to the end

Correct answer: B

B. A 45-minute duration warning threshold with notifications enabled for it, which alerts the team but leaves the run going

Explanation

Correct answer:

A 45-minute duration warning threshold with notifications enabled for it, which alerts the team but leaves the run going

Why this is correct:

A duration warning threshold signals when the run passes its expected 45-minute duration without terminating it. Enabling the duration-warning notification sends the requested email while leaving the run free to complete.

Why the other options are wrong:

A failure notification set on the job, which also fires for any run that exceeds its usual run duration by more than half an hour: Failure notifications are sent when a run ends unsuccessfully; they do not notify merely because a still-running job exceeds its usual duration. Use a duration threshold for a slow-but-running alert.

A 45-minute timeout set on the job with one retry, which emails the team at the time limit and then restarts the run so it can finish: When a run reaches its maximum timeout, the scheduler terminates it; retrying creates another attempt rather than letting the overlong run finish. Use a non-terminating duration warning instead.

A 45-minute timeout set on the job, which emails the team when the time limit is reached and then

lets the run carry on to the end: A maximum-duration limit stops a run when it is reached. The non-terminating choice is the warning threshold, which alerts while execution continues.

References:

1. Add notifications on a job: <https://docs.databricks.com/aws/en/jobs/notifications>

Question 14

A bronze **STRING** column **raw** holds **JSON** events whose fields change often, sometimes weekly. Silver must store the events without a fixed schema, and analysts need the nested **device.battery** value as a **DOUBLE**.

Which approach fits these requirements?

- A. Store **raw** directly as a **VARIANT** column **v** and read **v.device.battery** with dot notation, which navigates **VARIANT** values the same way it does for structs
- B. Store **parse_json(raw)** in a **VARIANT** column **v** and query **v:device.battery**, which already returns the nested battery value typed as a **DOUBLE** on its own
- C. Store **parse_json(raw)** output in a **VARIANT** column **v**, then extract **v:device.battery::double** at query time with an explicit type cast
- D. Parse **raw** with **from_json** using this week's schema, then read the battery field from the resulting parsed struct and cast that field to **DOUBLE**

Correct answer: C

C. Store **parse_json(raw)** output in a **VARIANT** column **v**, then extract **v:device.battery::double** at query time with an explicit type cast

Explanation

Correct answer:

Store **parse_json(raw)** output in a **VARIANT** column **v**, then extract **v:device.battery::double** at query time with an explicit type cast

Why this is correct:

parse_json converts the changing **JSON** text into a **VARIANT**, so the bronze representation does not require a fixed nested schema. The colon operator navigates to **device.battery**; casting that extracted value to **DOUBLE** gives silver the requested typed field while leaving the raw semi-structured value flexible.

Why the other options are wrong:

Parse raw with from_json using this week's schema, then read the battery field from the resulting parsed struct and cast that field to DOUBLE: **from_json** parses against a declared schema, so weekly field changes require schema maintenance and can leave new fields unavailable. This is useful when a stable typed struct is the desired output, not when bronze should remain schemaless.

Store parse_json(raw) in a VARIANT column v and query v:device.battery, which already returns the nested battery value typed as a DOUBLE on its own: Path extraction from **VARIANT** yields a variant value, not an automatically typed SQL double. The explicit cast is what establishes the output type.

Store raw directly as a VARIANT column v and read v.device.battery with dot notation, which navigates VARIANT values the same way it does for structs: Dot access is the usual syntax for struct fields; a VARIANT uses JSON-path access such as the colon operator. `parse_json` is also needed to convert the string into a variant value.

References:

1. **VARIANT type:** <https://docs.databricks.com/aws/en/sql/language-manual/data-types/variant-type>

Question 15

A COPY INTO load must consider only JSON files whose modification time is later than a known cutoff. Files already loaded should still be skipped by normal COPY INTO tracking.

```
FORMAT_OPTIONS ('_____ ' = '2026-08-01 00:00:00.000+00:00')
```

Which reader option expresses the cutoff?

- A. `modifiedAfter`, which restricts the load to those files whose timestamps are newer than the timestamp value that the option itself supplies
- B. `cutoffTimestamp`, which COPY INTO compares with each file's recorded creation time before deciding whether that file is loaded
- C. `pathGlobFilter`, which compares every file's modification time with the timestamp value that is written into its own glob pattern
- D. `modifiedBefore`, which restricts the load to those files whose timestamps come earlier than the timestamp value that the option itself supplies

Correct answer: A

A. `modifiedAfter`, which restricts the load to those files whose timestamps are newer than the timestamp value that the option itself supplies

Explanation

Correct answer:

`modifiedAfter`, which restricts the load to those files whose timestamps are newer than the timestamp value that the option itself supplies

Why this is correct:

`modifiedAfter` is a reader option that filters files by modification timestamp. Setting the cutoff in `FORMAT_OPTIONS` limits the candidate JSON files to newer modifications; COPY INTO's independent idempotency tracking still skips files already loaded.

Why the other options are wrong:

`pathGlobFilter`, which compares every file's modification time with the timestamp value that is written into its own glob pattern: `pathGlobFilter` matches filenames, not file metadata such as modification time. A timestamp cutoff cannot be expressed as a filename glob unless the naming scheme itself encodes time.

`cutoffTimestamp`, which COPY INTO compares with each file's recorded creation time before deciding whether that file is loaded: `cutoffTimestamp` is not the applicable file-reader option. The supported timestamp filter is `modifiedAfter`, and it tests modification time.

modifiedBefore, which restricts the load to those files whose timestamps come earlier than the timestamp value that the option itself supplies: **modifiedBefore** selects older files, the opposite of the required later-than cutoff. It would exclude the recent files the load should include.

References:

1. Spark API options reference: <https://docs.databricks.com/aws/en/spark/api-options>

Question 16

A column mask policy on catalog finance masks columns tagged pii for all account users.

Members of the auditors group must see unmasked values, the rule must remain one policy, and the shared mask function must not change.

What should change?

- A. Create an additional policy for auditors with an unmasking function, which wins over the first as the more recent policy
- B. Change the shared mask function to return the raw values to members of auditors, so the existing policy then stops masking them
- C. Grant auditors ALL PRIVILEGES on the catalog itself, letting the holders of that privilege bypass the column mask policy
- D. Add an EXCEPT clause that names auditors to the existing policy, so its principals list then leaves that group out of masking

Correct answer: D

D. Add an EXCEPT clause that names auditors to the existing policy, so its principals list then leaves that group out of masking

Explanation

Correct answer:

Add an EXCEPT clause that names auditors to the existing policy, so its principals list then leaves that group out of masking

Why this is correct:

The ABAC policy's EXCEPT list exempts named principals from an otherwise matching policy. Adding auditors to the existing policy's exception list preserves one policy and leaves the shared mask function unchanged while removing this group's records from policy application.

Why the other options are wrong:

Create an additional policy for auditors with an unmasking function, which wins over the first as the more recent policy: Policy precedence is not determined by which policy was created most recently, and overlapping mask policies are not a safe unmasking mechanism. The existing policy supports explicit exemptions.

Change the shared mask function to return the raw values to members of auditors, so the existing policy then stops masking them: Changing the shared function changes behavior for every principal covered by the policy and violates the requirement to keep that function unchanged. An exception scopes the relief to auditors.

Grant auditors ALL PRIVILEGES on the catalog itself, letting the holders of that privilege bypass the column mask policy: Broad Unity Catalog privileges do not bypass a row or column policy. The policy's principal applicability must be adjusted directly.

References:

1. ABAC GRANT policies:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/grant-policies>

Question 17

A pipeline must create a streaming DataFrame from the Kafka topic `parcel_events`. Broker addresses and authentication settings are already held in the Python dictionary `broker_options`.

Which code reads the topic?

A. `spark.readStream.format("cloudFiles").options(broker_options).option("cloudFiles.format", "kafka").load()`

B. `spark.readStream.format("kafka").options(broker_options).option("topic", "parcel_events").load()`

C. `spark.readStream.format("kafka").options(broker_options).option("subscribe", "parcel_events").load()`

D. `spark.read.format("kafka").options(broker_options).option("subscribe", "parcel_events").load()`

Correct answer: C

C. `spark.readStream.format("kafka").options(broker_options).option("subscribe", "parcel_events").load()`

Explanation

Correct answer:

```
spark.readStream.format("kafka").options(broker_options).option("subscribe", "parcel_events").load()
```

Why this is correct:

Kafka streaming reads use Structured Streaming's `readStream`, the Kafka source format, and a subscription selector such as `subscribe`. Passing the prepared broker options supplies connection and authentication settings while the topic option tells the source which stream to consume.

Why the other options are wrong:

`spark.readStream.format("kafka").options(broker_options).option("topic", "parcel_events").load()`: The batch read API returns a bounded DataFrame; Kafka offsets are read as a finite range rather than continuously. A live stream requires `readStream`.

`spark.read.format("kafka").options(broker_options).option("subscribe", "parcel_events").load()`: This combines batch reading with Kafka's subscription option, but subscription describes topic selection, not continuous execution. Switching to `readStream` makes it streaming.

`spark.readStream.format("cloudFiles").options(broker_options).option("cloudFiles.format",`

"kafka").load(): Auto Loader discovers files in object storage and expects file-source options. Kafka is a dedicated Structured Streaming source and does not become one by setting `cloudFiles.format`.

References:

1. **Connect to Apache Kafka:** <https://docs.databricks.com/aws/en/connect/streaming/kafka>

Question 18

In a job, the report task depends on load and enrich and uses the Run if condition All done. For today's run, enrich was disabled in the job settings, and load succeeded.

What does the run show for enrich and report?

- A. enrich runs anyway because disabling applies only to scheduled runs, and report then runs once both of its parents finish running
- B. enrich shows as Skipped and report fails, because a disabled parent is treated as a failed dependency under the All done condition
- C. enrich shows as Excluded and report is excluded as well, because All done needs every one of its parents to actually run first
- D. enrich finishes with a Disabled termination without ever running, and report runs because a disabled parent counts as being done

Correct answer: D

D. enrich finishes with a Disabled termination without ever running, and report runs because a disabled parent counts as being done

Explanation

Correct answer:

enrich finishes with a Disabled termination without ever running, and report runs because a disabled parent counts as being done

Why this is correct:

A disabled task is intentionally not executed and has a Disabled outcome. Lakeflow Jobs treats a disabled dependency as done when evaluating All done, so after load succeeds, report runs once both dependencies are considered complete.

Why the other options are wrong:

enrich runs anyway because disabling applies only to scheduled runs, and report then runs once both of its parents finish running: Disabling a task applies to job runs generally, not only scheduled runs. The disabled task does not execute for this run.

enrich shows as Skipped and report fails, because a disabled parent is treated as a failed dependency under the All done condition: Disabled is a distinct task outcome, not a failure. The documented All done behavior treats a disabled parent as done, so the report condition is satisfied.

enrich shows as Excluded and report is excluded as well, because All done needs every one of its parents to actually run first: All done does not require each dependency to have executed its task logic. A disabled dependency is considered done for downstream evaluation.

References:

1. Disabled tasks in Lakeflow Jobs: <https://docs.databricks.com/gcp/en/jobs/disabled-tasks>

Question 19

Table a holds regions 'US' and NULL. Table b holds 'US', NULL, and NULL. An engineer runs the query below to count matching rows.

```
SELECT COUNT(*) FROM a JOIN b ON a.region <=> b.region;
```

What count does the query return?

- A. An error, because the null-safe equality operator is not allowed in a join condition and can be used only in a WHERE clause filter
- B. 2, because the <=> operator matches each NULL row at most once, pairing a's NULL row with just one of b's NULL rows plus the US match
- C. 1, because a NULL value never equals another NULL, so only the two US rows can match each other in the join, giving a single output row
- D. 3, because the <=> operator treats two NULLs as equal, so a's NULL row matches both of b's NULL rows plus the US match

Correct answer: D

D. 3, because the <=> operator treats two NULLs as equal, so a's NULL row matches both of b's NULL rows plus the US match

Explanation

Correct answer:

3, because the <=> operator treats two NULLs as equal, so a's NULL row matches both of b's NULL rows plus the US match

Why this is correct:

<=> is null-safe equality: it treats NULL as equal to NULL. The US rows yield one matching pair, and the single NULL in table a pairs with both NULL rows in b, producing three joined rows total.

Why the other options are wrong:

2, because the <=> operator matches each NULL row at most once, pairing a's NULL row with just one of b's NULL rows plus the US match: Null-safe equality does not enforce one-to-one pairing. A join emits every matching pair, so one NULL row on the left matches both NULL rows on the right.

1, because a NULL value never equals another NULL, so only the two US rows can match each other in the join, giving a single output row: Ordinary = does not evaluate true for two NULLs, but <=> specifically changes that rule. The NULL rows therefore contribute matches in addition to US.

An error, because the null-safe equality operator is not allowed in a join condition and can be used only in a WHERE clause filter: <=> is a supported null-safe comparison and can be used in join

predicates. It is not restricted to filtering clauses.

References:

1. **equal_null function:** https://docs.databricks.com/aws/en/sql/language-manual/functions/equal_null

Question 20

A dashboard shows the number of distinct `visitor_id` values in a 40-billion-row table and refreshes every minute. Exact counting has been too slow for that interval, and a small relative error is acceptable.

Which expression fits?

- A. `COUNT(DISTINCT visitor_id)`
- B. `COUNT(DISTINCT visitor_id) FILTER (WHERE visitor_id IS NOT NULL)`
- C. `SIZE(COLLECT_SET(visitor_id))`
- D. `APPROX_COUNT_DISTINCT(visitor_id)`

Correct answer: D

D. `APPROX_COUNT_DISTINCT(visitor_id)`

Explanation

Correct answer:

`APPROX_COUNT_DISTINCT(visitor_id)`

Why this is correct:

`APPROX_COUNT_DISTINCT` estimates the number of distinct non-null visitor IDs using a probabilistic sketch, trading a controlled relative error for much faster aggregation at very large scale. That is the stated tradeoff for a dashboard refreshing every minute.

Why the other options are wrong:

`SIZE(COLLECT_SET(visitor_id))`: `COLLECT_SET` materializes the unique IDs into a collection before measuring its size, which can create a huge intermediate result and memory pressure. It does not provide the scalable approximate count requested.

`COUNT(DISTINCT visitor_id) FILTER (WHERE visitor_id IS NOT NULL)`: `COUNT(DISTINCT ...)` remains exact despite the filter, so it retains the expensive full distinct aggregation that caused the refresh-time problem.

`COUNT(DISTINCT visitor_id)`: This is the exact distinct aggregate; its correctness is useful when no approximation is allowed, but the question explicitly permits a small relative error to improve performance.

References:

1. `approx_count_distinct` aggregate function:

https://docs.databricks.com/aws/en/sql/language-manual/functions/approx_count_distinct

Question 21

Several notebook tasks in one job need the same `reporting_date` value. Operators must be able to override it for a single manual run, and the team wants one default kept in the job configuration.

Where should `reporting_date` be defined?

- A. As a job parameter with a default value, which every task receives and which an operator can replace when starting a run of the job
- B. As a task value set by the first task in the job, which later tasks read and which operators can change in the Jobs UI before a manual run
- C. As a separate task parameter on each notebook task, each one holding the same default for operators to edit task by task before their runs
- D. As a widget default set in each notebook, which operators change by editing the notebooks before they start a manual run

Correct answer: A

A. As a job parameter with a default value, which every task receives and which an operator can replace when starting a run of the job

Explanation

Correct answer:

As a job parameter with a default value, which every task receives and which an operator can replace when starting a run of the job

Why this is correct:

Job parameters provide shared key-value inputs with job-level defaults and can be overridden for an individual run. They are pushed down to notebook tasks, so each task can receive the same `reporting_date` while operators change it once in the run dialog.

Why the other options are wrong:

As a widget default set in each notebook, which operators change by editing the notebooks before they start a manual run: Notebook widgets provide parameter access inside a notebook, but editing each notebook default is not a convenient per-run override shared across the job. Job parameters centralize that value.

As a separate task parameter on each notebook task, each one holding the same default for operators to edit task by task before their runs: Duplicating the default on each task creates multiple settings to keep aligned and makes overrides task-by-task. A job parameter supplies one shared input.

As a task value set by the first task in the job, which later tasks read and which operators can change in the Jobs UI before a manual run: Task values pass outputs generated during a job run from

one task to another; they are not operator-editable job inputs configured before a run starts.

References:

1. **Configure job parameters:** <https://docs.databricks.com/aws/en/jobs/job-parameters>

Question 22

A CI pipeline authenticated as `ci_deployer` deploys a bundle to the prod target. The prod jobs must execute as the service principal `sp_etl` rather than as whichever identity performed the deployment.

What should the prod target configure?

- A. A `run_as` mapping that explicitly names `sp_etl`, so every job in the target executes with that identity instead of the deploying one
- B. A custom variable named `run_identity` set to `sp_etl`, which the deploy command then applies to every job in the target as both its owner and runner
- C. `mode: production` on the target, which switches each deployed job's run identity over to the workspace's own default principal
- D. A permissions entry giving `sp_etl` `CAN_MANAGE` on the deployed jobs, which makes each job run under that principal from then on

Correct answer: A

A. A `run_as` mapping that explicitly names `sp_etl`, so every job in the target executes with that identity instead of the deploying one

Explanation

Correct answer:

A `run_as` mapping that explicitly names `sp_etl`, so every job in the target executes with that identity instead of the deploying one

Why this is correct:

`run_as` separates the identity that deploys the bundle from the identity that executes its jobs. Setting the production target's run identity to service principal `sp_etl` makes job permissions and actions operate under that principal rather than under `ci_deployer`.

Why the other options are wrong:

mode: production on the target, which switches each deployed job's run identity over to the workspace's own default principal: Production deployment mode affects deployment behavior and resource naming; it does not by itself choose the runtime identity. The identity must be declared explicitly.

A permissions entry giving `sp_etl` `CAN_MANAGE` on the deployed jobs, which makes each job run under that principal from then on: `CAN_MANAGE` lets a principal administer a job, but it does not make that principal the job's run identity. Access control and execution identity are separate settings.

A custom variable named `run_identity` set to `sp_etl`, which the deploy command then applies to every job in the target as both its owner and runner: A custom bundle variable only supplies a value

where referenced; it has no automatic meaning to the job service. `run_as` is the supported identity configuration.

References:

1. Specify a run identity for a Declarative Automation Bundles workflow:

<https://docs.databricks.com/gcp/en/dev-tools/bundles/run-as>

Question 23

A pipeline table must stop its update whenever a record has a null event_id. Invalid records must not be silently dropped or merely counted in the metrics.

```
CONSTRAINT valid_event EXPECT (event_id IS NOT NULL)
```

What should be added to the constraint?

- A.** ON VIOLATION FAIL UPDATE, which stops the update as soon as any such record appears, so none of that update is committed
- B.** ON VIOLATION FAIL ROW, which fails only the individual records that break the rule while the rest of the update commits to the table
- C.** ON VIOLATION DROP ROW, which rejects the whole update after first removing each individual record that breaks the rule from it
- D.** Nothing further is needed, since an expectation without a violation clause rejects the whole update whenever any record breaks it

Correct answer: A

A. ON VIOLATION FAIL UPDATE, which stops the update as soon as any such record appears, so none of that update is committed

Explanation

Correct answer:

ON VIOLATION FAIL UPDATE, which stops the update as soon as any such record appears, so none of that update is committed

Why this is correct:

A pipeline expectation with ON VIOLATION FAIL UPDATE fails the pipeline update if any record violates its predicate, preventing that update from committing invalid rows. The event_id IS NOT NULL check is evaluated per record, so a missing ID stops the update instead of merely changing that row.

Why the other options are wrong:

ON VIOLATION DROP ROW, which rejects the whole update after first removing each individual record that breaks the rule from it: Keeping the valid subset would commit an update after discarding the bad records, but the requirement says any violation must stop the entire update. FAIL UPDATE provides that all-or-nothing behavior.

Nothing further is needed, since an expectation without a violation clause rejects the whole update whenever any record breaks it: An expectation without an action records metrics but does not fail or drop violating rows. The failure action must be declared explicitly.

ON VIOLATION FAIL ROW, which fails only the individual records that break the rule while the rest of the update commits to the table: FAIL ROW is not the supported expectation action; the pipeline controls behavior with DROP ROW, FAIL UPDATE, or no action. To stop the update, use FAIL UPDATE.

References:

1. Manage data quality with pipeline expectations:

<https://docs.databricks.com/gcp/en/ldp/expectations>

Question 24

The function `mask_ssn(ssn STRING, region STRING)` returns `ssn` unmasked only to the `eu_hr` group on EU rows and a masked value otherwise. It must mask `ssn` on `hr.people`, receiving each row's region.

Which statement applies it?

- A. `ALTER TABLE hr.people ALTER COLUMN ssn SET MASK mask_ssn USING COLUMNS (region);`
- B. `ALTER TABLE hr.people ALTER COLUMN ssn SET MASK mask_ssn(ssn, region);`
- C. `ALTER TABLE hr.people SET ROW FILTER mask_ssn ON (ssn, region);`
- D. `ALTER TABLE hr.people ALTER COLUMN region SET MASK mask_ssn USING COLUMNS (ssn);`

Correct answer: A

- A. `ALTER TABLE hr.people ALTER COLUMN ssn SET MASK mask_ssn USING COLUMNS (region);`

Explanation

Correct answer:

`ALTER TABLE hr.people ALTER COLUMN ssn SET MASK mask_ssn USING COLUMNS (region);`

Why this is correct:

A column mask is attached to the column whose values should be transformed. `USING COLUMNS (region)` supplies the additional row attribute to the function, while `ssn` is passed as the masked column's argument automatically; this binds the two-argument mask to the correct field.

Why the other options are wrong:

`ALTER TABLE hr.people SET ROW FILTER mask_ssn ON (ssn, region);`: A row filter returns a Boolean decision about whether a whole row is visible. This function returns an SSN value for masking, so it belongs on a column instead.

`ALTER TABLE hr.people ALTER COLUMN region SET MASK mask_ssn USING COLUMNS (ssn);`: This attaches the mask to `region`, whose type and values are not the sensitive SSN output. The policy must mask `ssn` and pass `region` as the additional input.

`ALTER TABLE hr.people ALTER COLUMN ssn SET MASK mask_ssn(ssn, region);`: The mask binding syntax uses `USING COLUMNS` to provide extra function arguments; calling the function directly in `SET MASK` is not the column-mask attachment form.

References:

1. Manually apply row filters and column masks:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply>

Question 25

A join is planned as sort-merge because stale statistics overestimate one input. At runtime that input's shuffle output turns out to be only 5 MB. AQE is enabled with its default settings.

What can AQE do here?

- A. Replan the join as a broadcast hash join based on the measured runtime size, with no change needed to the query text or its hints
- B. Nothing, because AQE's runtime changes cover partition coalescing and skew splitting but not a join's strategy
- C. Convert it to a shuffled hash join only, because once a shuffle has already run AQE can no longer switch that join to broadcast
- D. Cache the small input on each executor for later queries to reuse, with AQE promoting small shuffle outputs into cached tables

Correct answer: A

A. Replan the join as a broadcast hash join based on the measured runtime size, with no change needed to the query text or its hints

Explanation

Correct answer:

Replan the join as a broadcast hash join based on the measured runtime size, with no change needed to the query text or its hints

Why this is correct:

AQE can revise a sort-merge join after shuffle statistics reveal the actual input size. When the runtime size falls below the adaptive broadcast threshold, it can convert the join to a broadcast hash join without editing the SQL or supplying a hint.

Why the other options are wrong:

Cache the small input on each executor for later queries to reuse, with AQE promoting small shuffle outputs into cached tables: Broadcasting a join input is a per-query execution strategy, not a reusable cache operation. Caching would require a separate cache action and would not by itself correct this planned join.

Nothing, because AQE's runtime changes cover partition coalescing and skew splitting but not a join's strategy: AQE does more than coalescing and skew splitting; it can also adapt join strategies using measured shuffle statistics. The observed 5 MB relation supplies the information missing from stale estimates.

Convert it to a shuffled hash join only, because once a shuffle has already run AQE can no longer

switch that join to broadcast: AQE can change a sort-merge join to broadcast after materializing shuffle statistics when the conditions are met. The fact that shuffle work has begun does not categorically prevent this adaptation.

References:

1. AQE Demo | Databricks: <https://docs.databricks.com/aws/en/notebooks/source/aqe-demo.html>

Question 26

`sales.cases` has a `region` column. Members of the supervisors group must see every row, and all other readers may see only rows where `region = 'CA'`. The function body below will be attached to the table as a row filter on `region`.

Which function body implements this rule?

- A. `RETURN IF(is_account_group_member('supervisors'), region = 'CA', TRUE);`
- B. `RETURN is_account_group_member('supervisors') AND region = 'CA';`
- C. `RETURN is_account_group_member('supervisors') OR region = 'CA';`
- D. `RETURN NOT is_account_group_member('supervisors') OR region = 'CA';`

Correct answer: C

C. `RETURN is_account_group_member('supervisors') OR region = 'CA';`

Explanation

Correct answer:

`RETURN is_account_group_member('supervisors') OR region = 'CA';`

Why this is correct:

The filter function must return `TRUE` for rows visible to the reader. Supervisors satisfy the group-membership test and therefore pass every row; other readers pass only when the row's region is CA. The `OR` expresses these two allowed cases.

Why the other options are wrong:

`RETURN NOT is_account_group_member('supervisors') OR region = 'CA';`: Negating the membership test gives supervisors the CA-only branch and admits every non-supervisor row through the first condition. It reverses who receives broad visibility.

`RETURN IF(is_account_group_member('supervisors'), region = 'CA', TRUE);`: This conditional returns `TRUE` for non-supervisors regardless of region and restricts supervisors to CA. Both outcomes are opposite the requested access rule.

`RETURN is_account_group_member('supervisors') AND region = 'CA';`: `AND` requires a reader to be in supervisors and the row to be CA. That excludes supervisors' non-CA records and excludes every non-supervisor.

References:

1. Row filters and column masks:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks>

Question 27

A DataFrame `df` has the columns `sku` and `unit_cost`. An engineer runs the line below and then inspects both DataFrames.

```
renamed = df.withColumnRenamed("unit_cost", "cost")
```

Which statement describes the two schemas afterward?

- A. `renamed` contains both `unit_cost` and `cost`, while `df` keeps only `unit_cost`, because renaming copies the existing column under a new name
- B. Neither one changes until an action runs, because renaming is a lazy transformation that leaves both schemas as they were before
- C. Both `df` and `renamed` have `cost` in place of `unit_cost`, because `withColumnRenamed` updates the DataFrame it is called on in place
- D. `renamed` has `cost` where `unit_cost` was, while `df` still has `unit_cost`, because DataFrame methods return new DataFrame objects

Correct answer: D

D. `renamed` has `cost` where `unit_cost` was, while `df` still has `unit_cost`, because DataFrame methods return new DataFrame objects

Explanation

Correct answer:

`renamed` has `cost` where `unit_cost` was, while `df` still has `unit_cost`, because DataFrame methods return new DataFrame objects

Why this is correct:

Spark DataFrames are immutable: transformations return a new DataFrame with a changed logical plan rather than editing the original object. `renamed` therefore has the new column name, while `df` continues to describe the original schema.

Why the other options are wrong:

`renamed` contains both `unit_cost` and `cost`, while `df` keeps only `unit_cost`, because renaming copies the existing column under a new name: `withColumnRenamed` replaces the column name in the returned schema; it does not add a second copy of the data under a new name. The original DataFrame remains separately available.

Both `df` and `renamed` have `cost` in place of `unit_cost`, because `withColumnRenamed` updates the DataFrame it is called on in place: Spark transformations do not mutate an existing DataFrame in place. The returned DataFrame must be assigned, as the code does, to make the `renamed` schema accessible.

Neither one changes until an action runs, because renaming is a lazy transformation that leaves both schemas as they were before: The schema transformation is represented lazily, but schema metadata on the returned DataFrame reflects the rename immediately. An action is needed to compute rows, not to establish the schema.

References:

1. PySpark on Databricks: <https://docs.databricks.com/aws/en/pyspark>

Question 28

One task in a multi-task job occasionally hangs while pulling data. That task must fail after 20 minutes, while the job as a whole may run longer when its remaining work is progressing normally.

Which setting should be configured?

- A. A 20-minute timeout set in that task's own settings, which fails it after 20 minutes while the job's overall limit stays separate from it
- B. A 20-minute duration warning threshold set on the job, which stops that task as soon as the duration warning is raised for it
- C. A 20-minute minimum retry interval on that task, which then cancels any attempt of the task that runs longer than the retry interval
- D. A 20-minute timeout set on the whole job, which fails the run once all of its tasks together have run for longer than 20 minutes

Correct answer: A

A. A 20-minute timeout set in that task's own settings, which fails it after 20 minutes while the job's overall limit stays separate from it

Explanation

Correct answer:

A 20-minute timeout set in that task's own settings, which fails it after 20 minutes while the job's overall limit stays separate from it

Why this is correct:

A task-level timeout caps the duration of that one task and marks it timed out if it exceeds 20 minutes. This isolates the stuck data pull while allowing other work and the overall job to continue under a longer job-level duration limit.

Why the other options are wrong:

A 20-minute duration warning threshold set on the job, which stops that task as soon as the duration warning is raised for it: A duration warning is an alert threshold, not a cancellation timer. The task continues after a warning unless a maximum completion timeout is configured.

A 20-minute minimum retry interval on that task, which then cancels any attempt of the task that runs longer than the retry interval: A retry interval controls the delay before another attempt after a failure; it does not limit how long the current attempt may run. Timeout is the per-attempt duration control.

A 20-minute timeout set on the whole job, which fails the run once all of its tasks together have run for longer than 20 minutes: A job-level timeout applies to the whole run, so productive tasks can be

cut off along with the hanging task. Configure the limit on the task that needs it.

References:

1. **Configure and edit tasks in Lakeflow Jobs:** <https://docs.databricks.com/aws/en/jobs/configure-task>

Question 29

A notebook needs daily order totals from a PostgreSQL table with two billion rows. Only the aggregated rows may cross the network, and the database can run the aggregation itself.

```
spark.read.format("jdbc").option("url", jdbc_url).option("_____",  
totals_sql).load()
```

Which option name belongs in the blank, given that `totals_sql` holds a **SELECT** with a **GROUP BY**?

- A.** `numPartitions`, which spreads the data transfer across several connections and then applies `totals_sql` to the fetched rows inside Spark
- B.** `dbtable`, which accepts only a plain table name, so Spark first reads the whole table and then pushes the **GROUP BY** to PostgreSQL by default
- C.** `query`, which runs `totals_sql` inside PostgreSQL itself and returns only the resulting output of its **GROUP BY** over the connection
- D.** `customSchema`, which applies `totals_sql` as the set of column types Spark then uses when it reads the full table over a single connection

Correct answer: C

C. `query`, which runs `totals_sql` inside PostgreSQL itself and returns only the resulting output of its **GROUP BY** over the connection

Explanation

Correct answer:

`query`, which runs `totals_sql` inside PostgreSQL itself and returns only the resulting output of its **GROUP BY** over the connection

Why this is correct:

The `JDBC` query option submits the **SELECT** in the source database's SQL dialect. PostgreSQL performs the **GROUP BY** before returning results, so only the aggregated totals cross the network rather than two billion source rows.

Why the other options are wrong:

`customSchema`, which applies `totals_sql` as the set of column types Spark then uses when it reads the full table over a single connection: `customSchema` declares how returned columns should be typed; it does not run a SQL statement. It cannot push this aggregation to PostgreSQL.

`dbtable`, which accepts only a plain table name, so Spark first reads the whole table and then pushes the **GROUP BY to PostgreSQL by default:** `dbtable` accepts a table or a parenthesized subquery, but it is not limited to a plain table name. More importantly, the stated **SELECT** belongs in

query, which is explicitly executed by the source database.

numPartitions, which spreads the data transfer across several connections and then applies totals_sql to the fetched rows inside Spark: Parallel read splitting is controlled by partitioning parameters, not by passing a SQL query. Running the aggregation after transfer would send far more data across the connection.

References:

1. JDBC connection: <https://docs.databricks.com/aws/en/connect/jdbc-connection>

Question 30

A cleanup task depends on read_feed and validate_feed and uses the Run if condition All done. In today's run read_feed succeeded and validate_feed failed, and no task was canceled, disabled, or excluded.

What happens to the cleanup task?

- A.** It runs, because All done requires only that both of its dependencies have finished running, whatever final result each of them reports back
- B.** It is marked Upstream failed, because a failed dependency blocks downstream tasks whatever Run if condition they use in the job
- C.** It is excluded from the run, because All done runs a task only when no dependency failed and at least one of the dependencies actually ran
- D.** It runs twice in the job, once after each dependency finishes, because All done evaluates each completed dependency separately as it ends

Correct answer: A

A. It runs, because All done requires only that both of its dependencies have finished running, whatever final result each of them reports back

Explanation

Correct answer:

It runs, because All done requires only that both of its dependencies have finished running, whatever final result each of them reports back

Why this is correct:

All done waits until every dependency has finished, then runs without requiring any particular result state. Because both feed tasks have ended, the cleanup task is eligible even though validation failed; this condition is useful for cleanup and finalization tasks that must run after success or failure.

Why the other options are wrong:

It is excluded from the run, because All done runs a task only when no dependency failed and at least one of the dependencies actually ran: This describes a "none failed" style condition, which would exclude the case where validate_feed failed. All done intentionally ignores whether dependencies succeeded.

It runs twice in the job, once after each dependency finishes, because All done evaluates each completed dependency separately as it ends: A downstream task is scheduled after its dependencies are complete as a group, not once for each dependency completion. It runs once after both upstream tasks finish.

It is marked Upstream failed, because a failed dependency blocks downstream tasks whatever Run if condition they use in the job: A failed parent does not automatically block every downstream task. The selected Run if condition controls whether the failure matters, and All done accepts it.

References:

- 1. Configure task dependencies:** <https://docs.databricks.com/gcp/en/jobs/run-if>

Question 31

An engineer wants a named query over existing tables for use only in her current notebook session. It must persist no data and create no object that other users can see.

Which object fits?

- A. A global temporary view, which stores no data and stays hidden from other users for as long as the attached compute resource runs
- B. A standard view created in the engineer's personal schema, which other users cannot see until someone shares that personal schema with them
- C. A materialized view, which stores nothing until its first refresh and remains visible only to the single user who originally created it
- D. A temporary view, which disappears when her notebook later detaches or restarts, and it holds no stored results of its own anywhere

Correct answer: D

D. A temporary view, which disappears when her notebook later detaches or restarts, and it holds no stored results of its own anywhere

Explanation

Correct answer:

A temporary view, which disappears when her notebook later detaches or restarts, and it holds no stored results of its own anywhere

Why this is correct:

A temporary view registers a named query for the current Spark session without materializing its results. It is available to work in that session and is removed when the session ends, matching the notebook-scoped, non-persistent use case.

Why the other options are wrong:

A materialized view, which stores nothing until its first refresh and remains visible only to the single user who originally created it: A materialized view stores query results and has catalog-level lifecycle and access controls. It is intended for persisted, refreshable results, not a private session alias.

A global temporary view, which stores no data and stays hidden from other users for as long as the attached compute resource runs: A global temporary view is shared across sessions attached to the Spark application through a special namespace; it is not hidden as a session-private object. The ordinary temporary view is local to the creating session.

A standard view created in the engineer's personal schema, which other users cannot see until someone shares that personal schema with them: A standard catalog view persists as a Unity

Catalog object and can be granted to other principals. A personal schema may restrict access, but it is still a persistent shared catalog object.

References:

1. CREATE VIEW | Databricks on AWS:

<https://docs.databricks.com/aws/pt/sql/language-manual/sql-ref-syntax-ddl-create-view>

Question 32

A job must start a new run as soon as each run ends, around the clock, and failed runs should be retried automatically with increasing delays between attempts.

Which trigger fits?

- A.** A continuous trigger, which then starts the next run once the current run has ended and backs off exponentially between its retries
- B.** A scheduled trigger every minute with one concurrent run allowed at a time, so that a new run begins almost as soon as the previous one ends
- C.** A file arrival trigger on the job's own output path, which then starts the next run when the current run finishes writing its results there
- D.** A table update trigger on the job's own target table, which starts the next run each time the current run commits its output to that table

Correct answer: A

A. A continuous trigger, which then starts the next run once the current run has ended and backs off exponentially between its retries

Explanation

Correct answer:

A continuous trigger, which then starts the next run once the current run has ended and backs off exponentially between its retries

Why this is correct:

Continuous mode schedules another run when the current run completes or fails, keeping the job active around the clock. Continuous jobs use exponential backoff when retrying failures, so retry attempts become less frequent as failures persist.

Why the other options are wrong:

A table update trigger on the job's own target table, which starts the next run each time the current run commits its output to that table: A table trigger responds to upstream table updates, not to the completion of the current job run. Monitoring the job's output table could also create an unnecessary feedback loop.

A scheduled trigger every minute with one concurrent run allowed at a time, so that a new run begins almost as soon as the previous one ends: A one-minute schedule starts on clock intervals, not immediately when a run ends. With concurrency limited to one, schedule ticks during a long run may be skipped rather than forming a completion-driven chain.

A file arrival trigger on the job's own output path, which then starts the next run when the current

run finishes writing its results there: File arrival triggers watch new files at a Unity Catalog location; they do not treat the job finishing as a special event. A continuously scheduled job has the required completion behavior.

References:

1. Run jobs continuously: <https://docs.databricks.com/gcp/en/jobs/continuous>

Question 33

A workspace runs its notebooks on classic compute. The security team asks where the Spark driver and executors process notebook data, and where the web application and the job scheduler run.

Which statement is accurate?

- A. Everything runs in the Databricks control plane, and classic compute in the customer's cloud account hosts only the storage that the queries read
- B. The driver and executors run in the Databricks-managed serverless compute plane itself, and the customer's own account holds the storage buckets
- C. The driver itself runs in the Databricks control plane while classic compute executors run in the customer's own cloud account, so query plans never leave Databricks
- D. The Spark processing happens inside the customer's own cloud account, while the Databricks control plane hosts the web application and the scheduler

Correct answer: D

D. The Spark processing happens inside the customer's own cloud account, while the Databricks control plane hosts the web application and the scheduler

Explanation

Correct answer:

The Spark processing happens inside the customer's own cloud account, while the Databricks control plane hosts the web application and the scheduler

Why this is correct:

Classic compute separates the workspace control plane from the plane that processes data. The driver's Spark execution and the executors run on classic compute resources in the customer's cloud account, while the Databricks web application and job scheduler are control-plane services.

Why the other options are wrong:

The driver itself runs in the Databricks control plane while classic compute executors run in the customer's own cloud account, so query plans never leave Databricks: The driver is part of the compute workload, not the control plane. Plans and data processing happen on classic compute in the customer's account.

The driver and executors run in the Databricks-managed serverless compute plane itself, and the customer's own account holds the storage buckets: That describes Databricks-managed serverless compute. The stem explicitly says these notebooks use classic compute.

Everything runs in the Databricks control plane, and classic compute in the customer's cloud

account hosts only the storage that the queries read: The control plane does not process all Spark workloads, and classic compute is where the driver and executors run, not just a place to store data.

References:

1. High-level architecture: <https://docs.databricks.com/aws/en/getting-started/high-level-architecture>

Question 34

A team creates a Lakeflow Connect managed ingestion pipeline for a SaaS source. The pipeline setup references a Unity Catalog connection object named `crm_conn` that an admin created earlier.

What does `crm_conn` provide to the pipeline?

- A. The table-level grants that analysts need, replacing the `SELECT` grant on each of the destination tables that the pipeline creates
- B. A workspace secret scope reference that the pipeline reads at run time, so the credentials live in the scope rather than in Unity Catalog
- C. The source's authentication details, which are stored centrally in Unity Catalog so that no secrets live in the pipeline definition itself
- D. A private network route to the SaaS API, which the connection object then opens through the workspace's private endpoint on each pipeline update

Correct answer: C

C. The source's authentication details, which are stored centrally in Unity Catalog so that no secrets live in the pipeline definition itself

Explanation

Correct answer:

The source's authentication details, which are stored centrally in Unity Catalog so that no secrets live in the pipeline definition itself

Why this is correct:

A Unity Catalog connection is a governed object for an external system that stores connection settings and authentication details. Referencing `crm_conn` lets the managed ingestion pipeline use those credentials centrally rather than embedding secrets in its own definition; network routing is configured separately.

Why the other options are wrong:

The table-level grants that analysts need, replacing the `SELECT` grant on each of the destination tables that the pipeline creates: A source connection authenticates to the external system; it does not grant analysts `SELECT` on the destination tables. Destination access remains governed by Unity Catalog table privileges.

A private network route to the SaaS API, which the connection object then opens through the workspace's private endpoint on each pipeline update: Connections hold endpoint and credential configuration, but they do not create private network routes or endpoints. Network policies and cloud networking determine connectivity.

A workspace secret scope reference that the pipeline reads at run time, so the credentials live in the scope rather than in Unity Catalog: The connection object is not a pointer to a workspace secret scope. Its managed-ingestion credentials are governed through Unity Catalog.

References:

1. **Unity Catalog connections:** <https://docs.databricks.com/aws/en/connect/uc-connections>

Question 35

A managed Delta table's query patterns shift often, and the team wants Databricks itself to choose and update the clustering keys over time. Predictive optimization is enabled for the table.

Which statement sets this up?

- A. `ALTER TABLE sales.orders CLUSTER BY (order_date, region);`
- B. `ALTER TABLE sales.orders SET TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true');`
- C. `ALTER TABLE sales.orders CLUSTER BY AUTO;`
- D. `OPTIMIZE sales.orders ZORDER BY AUTO;`

Correct answer: C

C. `ALTER TABLE sales.orders CLUSTER BY AUTO;`

Explanation

Correct answer:

`ALTER TABLE sales.orders CLUSTER BY AUTO;`

Why this is correct:

CLUSTER BY AUTO enables automatic selection of liquid-clustering keys for the table. With predictive optimization enabled, Databricks can manage the evolving layout rather than requiring the team to choose a fixed key list as access patterns change.

Why the other options are wrong:

OPTIMIZE sales.orders ZORDER BY AUTO;: ZORDER BY AUTO is not the automatic liquid-clustering configuration. Liquid clustering uses CLUSTER BY AUTO, and predictive optimization can maintain that layout.

ALTER TABLE sales.orders SET TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true');: Optimized writes affect file sizing during writes; they do not choose clustering keys or maintain data layout for shifting filters.

ALTER TABLE sales.orders CLUSTER BY (order_date, region);: An explicit key list makes the team responsible for selecting and changing the clustering columns. The stem asks Databricks to choose them automatically.

References:

1. Use liquid clustering for tables: <https://docs.databricks.com/aws/en/delta/clustering>

Question 36

An engineer must compare a job's run durations this week with the same job's durations five months ago. The job's run history in the Jobs UI no longer shows runs from that far back.

Where should the engineer get the older durations?

- A. From DESCRIBE HISTORY on the job's output table, whose operation metrics record the total duration of each individual job run over the months
- B. From the system.lakeflow.job_run_timeline table, whose recorded run periods can be aggregated into per-run durations across all of those months
- C. From the Spark UI event logs recorded for each run, which the workspace keeps indefinitely for completed job clusters and their tasks
- D. From the job's matrix view, which keeps each individual run and task duration for the whole lifetime of the job, across all the months since it was created

Correct answer: B

B. From the system.lakeflow.job_run_timeline table, whose recorded run periods can be aggregated into per-run durations across all of those months

Explanation

Correct answer:

From the system.lakeflow.job_run_timeline table, whose recorded run periods can be aggregated into per-run durations across all of those months

Why this is correct:

The Jobs system table system.lakeflow.job_run_timeline records run intervals and has a 365-day retention window, allowing historical durations to be calculated beyond the Jobs UI's displayed history. Summing each run's period durations produces its total elapsed duration for comparison across periods.

Why the other options are wrong:

From the Spark UI event logs recorded for each run, which the workspace keeps indefinitely for completed job clusters and their tasks: Spark event logs describe Spark execution details and are not a guaranteed, indefinitely retained record of every job's end-to-end run duration. The job timeline system table is the durable operational history intended for this analysis.

From DESCRIBE HISTORY on the job's output table, whose operation metrics record the total duration of each individual job run over the months: Delta table history records table operations, not a complete timeline for every job that may have written to it. One job may perform multiple operations, and other jobs may use different output tables.

From the job's matrix view, which keeps each individual run and task duration for the whole

lifetime of the job, across all the months since it was created: The UI's matrix view visualizes runs available in job history; it does not provide an unlimited archive. The stated five-month period is why the system table is needed.

References:

1. **Jobs system table reference:** <https://docs.databricks.com/aws/en/admin/system-tables/jobs>

Question 37

An Auto Loader stream infers the schema of CSV files that contain `order_id`, `amount`, and `order_ts`. The `cloudFiles.inferColumnTypes` option is left at its default, and `schemaHints` is set to `'amount DECIMAL(10,2)'`.

Which schema does Auto Loader infer?

- A. `order_id` as `BIGINT`, `order_ts` as `TIMESTAMP`, and `amount` as `DECIMAL(10,2)`, because Auto Loader samples the column values to type each unhinted column
- B. Every column as `STRING`, including `amount`, because schema hints take effect only when `cloudFiles.inferColumnTypes` is also set to `true` explicitly
- C. `amount` as `DECIMAL(10,2)` alone, because the schema hints replace inference entirely and drop every other column that the hint string does not name explicitly
- D. `order_id` and `order_ts` as `STRING` and `amount` as `DECIMAL(10,2)`, because the unhinted CSV columns default to `STRING` when their types are not inferred

Correct answer: D

D. `order_id` and `order_ts` as `STRING` and `amount` as `DECIMAL(10,2)`, because the unhinted CSV columns default to `STRING` when their types are not inferred

Explanation

Correct answer:

`order_id` and `order_ts` as `STRING` and `amount` as `DECIMAL(10,2)`, because the unhinted CSV columns default to `STRING` when their types are not inferred

Why this is correct:

Auto Loader defaults to `STRING` for CSV columns when `cloudFiles.inferColumnTypes` is `false`, avoiding unreliable guesses from sample values. A schema hint overrides the inferred type for `amount` while leaving other inferred columns in the schema as strings; hints do not replace the complete schema.

Why the other options are wrong:

amount as `DECIMAL(10,2)` alone, because the schema hints replace inference entirely and drop every other column that the hint string does not name explicitly: The CSV header still contributes `order_id` and `order_ts`; the hint only adjusts `amount` instead of replacing the whole schema.

order_id as `BIGINT`, order_ts as `TIMESTAMP`, and amount as `DECIMAL(10,2)`, because Auto Loader samples the column values to type each unhinted column: Spark's general CSV reader can infer types from sample values, but Auto Loader's type inference is off by default. Setting `cloudFiles.inferColumnTypes` to `true` is required for value-based typing of unhinted CSV fields.

Every column as `STRING`, including amount, because schema hints take effect only when

cloudFiles.inferColumnTypes is also set to true explicitly: A hint is honored whether or not cloudFiles.inferColumnTypes is enabled. The default STRING behavior applies to unhinted columns, not to the explicitly hinted amount.

References:

1. Configure schema inference and evolution in Auto Loader:

<https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema>

Question 38

Two engineers share a workspace and the same team repository. One can push from her Git folder, but the other gets an authentication error on push, although both have write access in the Git provider.

What should be fixed?

- A.** A workspace admin must grant the second engineer CAN EDIT permission on his Git folder, which authorizes pushes from it to the provider
- B.** The second engineer must pull before pushing, as the Git provider answers pushes from outdated Git folders with an authentication error
- C.** The second engineer needs CAN MANAGE permission on the first engineer's Git folder, which lends him her stored Git credentials for his own pushes
- D.** The second engineer must add his own set of Git provider credentials, such as a personal access token, in his own user settings

Correct answer: D

D. The second engineer must add his own set of Git provider credentials, such as a personal access token, in his own user settings

Explanation

Correct answer:

The second engineer must add his own set of Git provider credentials, such as a personal access token, in his own user settings

Why this is correct:

Databricks Git folders use the credentials configured for each individual user when communicating with the Git provider. The second engineer should add their own provider credential in personal settings; workspace write access in the provider does not create a local authentication token for Databricks to use.

Why the other options are wrong:

The second engineer needs CAN MANAGE permission on the first engineer's Git folder, which lends him her stored Git credentials for his own pushes: Git credentials are user-specific and are not borrowed through folder permissions. Granting someone access to another person's Git folder does not authenticate that user's push.

A workspace admin must grant the second engineer CAN EDIT permission on his Git folder, which authorizes pushes from it to the provider: CAN EDIT governs access to the Git folder in the workspace; the Git provider separately authenticates the push. Folder permissions cannot supply provider credentials.

The second engineer must pull before pushing, as the Git provider answers pushes from outdated Git folders with an authentication error: Pulling can synchronize commits but cannot fix an authentication failure. If credentials are missing or invalid, the subsequent push remains unauthorized.

References:

1. Connect your Git provider to Databricks:

<https://docs.databricks.com/aws/en/repos/get-access-tokens-from-git-provider>

Question 39

Two workflows each need to run an existing validation job that has its own multi-task graph. The team wants a single copy of that job to maintain.

Which task should each workflow add?

- A. A Pipeline task that points directly at the validation job, so each workflow runs that job's tasks as part of a pipeline update
- B. A notebook task that calls `dbutils.notebook.run` on the validation job's first notebook, which then runs the remaining tasks of that job
- C. A copy of the validation job's tasks in each workflow, kept in sync between the two through a shared notebook library that each of them imports
- D. A Run Job task that references the existing validation job, so each workflow triggers that job and all of its tasks in their defined order

Correct answer: D

D. A Run Job task that references the existing validation job, so each workflow triggers that job and all of its tasks in their defined order

Explanation

Correct answer:

A Run Job task that references the existing validation job, so each workflow triggers that job and all of its tasks in their defined order

Why this is correct:

A Run Job task references an existing Lakeflow Job and starts that job as a unit, preserving its own task graph, dependencies, and run history. Each workflow can invoke the shared validation job, so the team maintains the validation DAG in one place.

Why the other options are wrong:

A copy of the validation job's tasks in each workflow, kept in sync between the two through a shared notebook library that each of them imports: Duplicating each task creates two independently maintained graphs; a shared notebook library reuses code but does not synchronize job definitions or dependencies.

A notebook task that calls `dbutils.notebook.run` on the validation job's first notebook, which then runs the remaining tasks of that job: `dbutils.notebook.run` invokes a notebook, not a Lakeflow Job. It does not automatically execute the job's remaining task graph or honor its task-level orchestration.

A Pipeline task that points directly at the validation job, so each workflow runs that job's tasks as part of a pipeline update: A Pipeline task starts a Lakeflow pipeline update, not a job. The existing

validation workflow should be invoked using the job-specific task type.

References:

1. Run Job task for jobs: <https://docs.databricks.com/aws/en/jobs/tasks/run-job>

Question 40

The function `non_eu(region STRING) RETURNS BOOLEAN` exists. Every table in `prod.customers` with a column tagged `geo_region`, including later tables, must hide EU rows from `us_analysts`.

```
CREATE POLICY hide_eu ON SCHEMA prod.customers
    _____ non_eu
    TO us_analysts
    FOR TABLES
    MATCH COLUMNS has_tag('geo_region') AS region
    USING COLUMNS (region);
```

Which clause belongs in the blank?

- A. ROW FILTER, which passes each matched column value to the function and leaves out any row for which the function call returns FALSE
- B. WHERE, which appends the function as an extra predicate to every query that `us_analysts` run against the tables in the schema
- C. COLUMN MASK, which replaces the tagged column value on EU rows, leaving `us_analysts` a masked value in place of the original data
- D. ROW ACCESS POLICY, which binds the function directly to the schema, so each table created there later inherits the same row behavior

Correct answer: A

A. ROW FILTER, which passes each matched column value to the function and leaves out any row for which the function call returns FALSE

Explanation

Correct answer:

ROW FILTER, which passes each matched column value to the function and leaves out any row for which the function call returns FALSE

Why this is correct:

An ABAC ROW FILTER policy evaluates the supplied Boolean function for each row and exposes only rows for which it returns TRUE. The matched `geo_region` column is passed as the function's region input, so `non_eu` hides EU rows across the tagged schema tables.

Why the other options are wrong:

WHERE, which appends the function as an extra predicate to every query that `us_analysts` run against the tables in the schema: WHERE is not the policy type in this CREATE POLICY syntax. The policy declares an access-control behavior that Unity Catalog applies during reads; it is not appended to user-authored SQL text.

COLUMN MASK, which replaces the tagged column value on EU rows, leaving us_analysts a masked value in place of the original data: A column mask changes a column's displayed value while retaining the row. This requirement is to omit EU rows, so the function's Boolean result must be used as a row filter.

ROW ACCESS POLICY, which binds the function directly to the schema, so each table created there later inherits the same row behavior: ROW ACCESS POLICY is not the ABAC policy clause here. Unity Catalog uses the policy type ROW FILTER for this row-level restriction.

References:

1. Create and manage row filter and column mask policies:

<https://docs.databricks.com/gcp/en/data-governance/unity-catalog/abac/policies>

Question 41

A CI job authenticates the Databricks CLI with a personal access token supplied only through environment variables. No configuration profile or other authentication method exists on the runner.

Which TWO environment variables must be set?

- A. DATABRICKS_ACCOUNT_ID, which holds the unique ID of the Databricks account that owns the target workspace
- B. DATABRICKS_TOKEN, which holds the personal access token string value that the CLI then presents to authenticate
- C. DATABRICKS_CLIENT_SECRET, which holds the OAuth secret value paired with a given service principal's client ID
- D. DATABRICKS_CLIENT_ID, which holds a service principal's application ID value for its OAuth machine-to-machine sign-in
- E. DATABRICKS_HOST, which holds the full workspace URL that the CLI sends each of its API requests to during the job

Correct answers: B, E

- B. DATABRICKS_TOKEN, which holds the personal access token string value that the CLI then presents to authenticate
- E. DATABRICKS_HOST, which holds the full workspace URL that the CLI sends each of its API requests to during the job

Explanation

Correct answer:

DATABRICKS_TOKEN, which holds the personal access token string value that the CLI then presents to authenticate

DATABRICKS_HOST, which holds the full workspace URL that the CLI sends each of its API requests to during the job

Why this is correct:

For personal-access-token authentication, the CLI needs the token in DATABRICKS_TOKEN and the workspace endpoint in DATABRICKS_HOST. Together they provide the bearer credential and the API destination; client ID and secret are used for OAuth service-principal authentication instead.

Why the other options are wrong:

DATABRICKS_CLIENT_ID, which holds a service principal's application ID value for its OAuth machine-to-machine sign-in: DATABRICKS_CLIENT_ID identifies a service principal for OAuth flows,

but the scenario explicitly uses a PAT and has no OAuth configuration. It does not replace the token variable.

DATABRICKS_ACCOUNT_ID, which holds the unique ID of the Databricks account that owns the **target workspace**: An account ID is used for account-level APIs and is not the workspace URL needed to direct workspace requests. The required host variable is separate.

DATABRICKS_CLIENT_SECRET, which holds the OAuth secret value paired with a given service principal's client ID: A client secret pairs with a client ID for OAuth authentication. Personal access token authentication uses DATABRICKS_TOKEN instead.

References:

1. Environment variables and fields for unified authentication:

<https://docs.databricks.com/aws/en/dev-tools/auth/env-vars>

Question 42

Yesterday an engineer dropped the managed table `ops.silver.shipments` by mistake. No table with that name has been created since, and the drop is still inside the recovery window.

Which statement brings the table back?

- A. `RESTORE TABLE ops.silver.shipments TO VERSION AS OF 0;`
- B. `ALTER TABLE ops.silver.shipments SET MANAGED;`
- C. `CREATE TABLE ops.silver.shipments DEEP CLONE ops.silver.shipments VERSION AS OF 1;`
- D. `UNDROP TABLE ops.silver.shipments;`

Correct answer: D

D. `UNDROP TABLE ops.silver.shipments;`

Explanation

Correct answer:

`UNDROP TABLE ops.silver.shipments;`

Why this is correct:

Unity Catalog retains dropped managed-table metadata and data during its recovery window. `UNDROP TABLE` restores that dropped object, preserving its table identity; the question says no same-name replacement exists and the drop is still recoverable.

Why the other options are wrong:

`ALTER TABLE ops.silver.shipments SET MANAGED;`: `SET MANAGED` converts an existing external table; it cannot recover an object that has been dropped from the catalog.

`RESTORE TABLE ops.silver.shipments TO VERSION AS OF 0;` `RESTORE` changes the contents of a table that still exists to a previous Delta version. It does not reverse a catalog drop.

`CREATE TABLE ops.silver.shipments DEEP CLONE ops.silver.shipments VERSION AS OF 1;` A deep clone needs a surviving source table to copy from. The named object was dropped, so this command cannot use it as its own clone source.

References:

1. Unity Catalog managed tables for Delta Lake and Apache Iceberg:

<https://docs.databricks.com/aws/en/tables/managed>

Question 43

An engineer tries `spark.readStream` with Spark's built-in JDBC source to capture inserts and updates from a relational database continuously. No separate change data capture connector is involved.

What is the result?

- A. Spark starts a streaming query that rereads the full table again on every trigger, so each micro-batch repeats the rows it already read
- B. The read works only when `numPartitions` is set, because streaming through JDBC needs several parallel connections to track changes
- C. The read is not supported, since the JDBC reader handles batch loads alone, so ongoing changes need a CDC-capable tool instead
- D. Spark streams new rows once an increasing key column is configured as `partitionColumn`, which it then uses as the streaming offset

Correct answer: C

C. The read is not supported, since the JDBC reader handles batch loads alone, so ongoing changes need a CDC-capable tool instead

Explanation

Correct answer:

The read is not supported, since the JDBC reader handles batch loads alone, so ongoing changes need a CDC-capable tool instead

Why this is correct:

Spark's built-in JDBC reader is a batch data source and does not implement a streaming offset or change-capture protocol. Continuous inserts and updates require a source connector that can track changes, such as a database CDC capability, rather than repeatedly treating the full table as a stream.

Why the other options are wrong:

The read works only when `numPartitions` is set, because streaming through JDBC needs several parallel connections to track changes: `numPartitions` parallelizes a bounded JDBC read; it does not add change tracking or streaming semantics. More connections cannot turn a batch source into a CDC source.

Spark starts a streaming query that rereads the full table again on every trigger, so each micro-batch repeats the rows it already read: Structured Streaming cannot accept the ordinary JDBC reader as an incremental stream source. It will not periodically rescan and deduplicate the whole table automatically.

Spark streams new rows once an increasing key column is configured as partitionColumn, which it then uses as the streaming offset: partitionColumn divides batch reads into ranges; it is not a streaming watermark or offset. A monotonically increasing key alone does not add JDBC streaming support.

References:

1. **Spark data sources:** <https://docs.databricks.com/aws/en/connect/spark-data-sources>

Question 44

Upstream pipelines commit to sales.orders dozens of times an hour at irregular moments, with no quiet periods during the day. A downstream job should react to those changes but must not start more than once every 30 minutes.

Which trigger setup fits?

- A. A scheduled trigger set to every 30 minutes, which starts the job on that fixed cadence even when orders has not changed since the prior job run
- B. A continuous trigger with a 30-minute pause between consecutive runs, checking orders for new commits each time the job restarts
- C. A table update trigger on orders with a 30-minute wait after last change, so a new run starts as soon as the commits to orders pause for that long
- D. A table update trigger on orders with a 30-minute minimum time between triggers, so any commits arriving sooner simply wait until that gap has fully passed

Correct answer: D

D. A table update trigger on orders with a 30-minute minimum time between triggers, so any commits arriving sooner simply wait until that gap has fully passed

Explanation

Correct answer:

A table update trigger on orders with a 30-minute minimum time between triggers, so any commits arriving sooner simply wait until that gap has fully passed

Why this is correct:

The table update trigger reacts to commits on sales.orders, while its minimum-time-between-triggers option enforces a cooldown between job runs. Updates during the cooldown remain eligible and can trigger after it expires, which caps run frequency without relying on quiet periods.

Why the other options are wrong:

A continuous trigger with a 30-minute pause between consecutive runs, checking orders for new commits each time the job restarts: Continuous mode starts another run after the prior run completes; it is not tied to commits on the specified table and therefore does not implement event-based change detection.

A table update trigger on orders with a 30-minute wait after last change, so a new run starts as soon as the commits to orders pause for that long: “Wait after last change” is a debounce: every new update resets the timer, so nonstop commits can prevent a run from starting. It is not the cooldown used to cap frequency.

A scheduled trigger set to every 30 minutes, which starts the job on that fixed cadence even when orders has not changed since the prior job run: A schedule fires at fixed times regardless of source changes. The requirement is to react to table updates and merely limit how often runs start.

References:

1. Trigger jobs when source tables are updated:

<https://docs.databricks.com/gcp/en/jobs/trigger-table-update>

Question 45

`corp.gold.daily_sales` is a Databricks SQL materialized view built from `corp.silver.sales_daily`. It must refresh on its own every hour, without a separate job to orchestrate it.

Which statement sets this up?

A. `CREATE OR REPLACE VIEW corp.gold.daily_sales SCHEDULE EVERY 1 HOUR AS SELECT * FROM corp.silver.sales_daily;`

B. `ALTER TABLE corp.gold.daily_sales SET TBLPROPERTIES ('refreshInterval' = '1 hour', 'source' = 'corp.silver.sales_daily');`

C. `REFRESH MATERIALIZED VIEW corp.gold.daily_sales EVERY 1 HOUR ON corp.silver.sales_daily;`

D. `CREATE OR REPLACE MATERIALIZED VIEW corp.gold.daily_sales SCHEDULE EVERY 1 HOUR AS SELECT * FROM corp.silver.sales_daily;`

Correct answer: D

D. `CREATE OR REPLACE MATERIALIZED VIEW corp.gold.daily_sales SCHEDULE EVERY 1 HOUR AS SELECT * FROM corp.silver.sales_daily;`

Explanation

Correct answer:

`CREATE OR REPLACE MATERIALIZED VIEW corp.gold.daily_sales SCHEDULE EVERY 1 HOUR AS SELECT * FROM corp.silver.sales_daily;`

Why this is correct:

A standalone materialized view definition can include a periodic refresh schedule. `SCHEDULE EVERY 1 HOUR` tells Databricks to refresh the cached query result on that interval, so a separate Lakeflow Job is unnecessary for orchestration.

Why the other options are wrong:

`ALTER TABLE corp.gold.daily_sales SET TBLPROPERTIES ('refreshInterval' = '1 hour', 'source' = 'corp.silver.sales_daily');`: A table property does not define a materialized view's query or its managed refresh schedule. The schedule belongs in the materialized-view DDL.

`REFRESH MATERIALIZED VIEW corp.gold.daily_sales EVERY 1 HOUR ON`

`corp.silver.sales_daily;`: `REFRESH MATERIALIZED VIEW` requests an individual refresh; `EVERY` is not appended as a recurring schedule to that command. Define the schedule in `CREATE` or `REPLACE`

MATERIALIZED VIEW.

CREATE OR REPLACE VIEW corp.gold.daily_sales SCHEDULE EVERY 1 HOUR AS SELECT * FROM corp.silver.sales_daily;; This creates a standard view, which computes from source data when queried and does not maintain cached results on an hourly refresh schedule.

References:

1. CREATE MATERIALIZED VIEW:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-create-materialized-view>

Question 46

A notebook task parameter references `output_schema`. The engineer deploys with `-t test` and supplies no `--var`, environment variable, or variable-file override.

```
variables:
  output_schema:
    default: scratch
targets:
  test:
    variables:
      output_schema: qa_shipments
```

Which value does `output_schema` resolve to?

- A. No value, because a variable that already declares a default cannot also be set inside a target, so the bundle's validation fails first
- B. `qa_shipments` at deploy time, but it is `scratch` whenever the job is later started with `bundle run`, which re-resolves variables from their defaults
- C. `qa_shipments`, because a target's own variables mapping overrides the top-level default setting whenever that particular target is selected
- D. `scratch`, because the top-level default always applies unless a different value is explicitly passed with `--var` on the deploy command line

Correct answer: C

C. `qa_shipments`, because a target's own variables mapping overrides the top-level default setting whenever that particular target is selected

Explanation

Correct answer:

`qa_shipments`, because a target's own variables mapping overrides the top-level default setting whenever that particular target is selected

Why this is correct:

Bundle variables resolve during deployment, and target-level variables override top-level defaults for the selected target. Since the deployment uses `-t test`, `qa_shipments` is the resolved value. This is an environment-specific setting, not a value recalculated when the deployed job later runs.

Why the other options are wrong:

scratch, because the top-level default always applies unless a different value is explicitly passed with `--var` on the deploy command line: A default supplies a value only when no more specific setting overrides it. The selected target provides its own `output_schema`, so the default `scratch` is not used.

qa_shipments at deploy time, but it is scratch whenever the job is later started with bundle run, which re-resolves variables from their defaults: bundle run starts the deployed resource and passes job parameters; it does not redeploy the bundle or resolve its variables again. Runtime overrides require job parameters.

No value, because a variable that already declares a default cannot also be set inside a target, so the bundle's validation fails first: A target can override a top-level variable default; the two declarations are a normal way to select environment-specific values.

References:

1. Substitutions and variables in Declarative Automation Bundles:

<https://docs.databricks.com/aws/en/dev-tools/bundles/variables>

Question 47

A daily COPY INTO loads CSV files into corp.bronze.orders. New vendor files add a discount column, and the load must add it to the table instead of failing or dropping it.

```
COPY INTO corp.bronze.orders
FROM '/Volumes/corp/landing/orders/'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true', 'mergeSchema' = 'true')
_____;
```

Which clause completes the command?

- A. COPY_OPTIONS ('mergeSchema' = 'true')
- B. FORMAT_OPTIONS ('inferSchema' = 'true')
- C. COPY_OPTIONS ('force' = 'true')
- D. COPY_OPTIONS ('overwriteSchema' = 'true')

Correct answer: A

A. COPY_OPTIONS ('mergeSchema' = 'true')

Explanation

Correct answer:

COPY_OPTIONS ('mergeSchema' = 'true')

Why this is correct:

For COPY INTO, FORMAT_OPTIONS controls how the source reader interprets its files, while COPY_OPTIONS controls how the target Delta load behaves. Target schema evolution requires COPY_OPTIONS ('mergeSchema' = 'true'), allowing the new discount field to be added to the table.

Why the other options are wrong:

COPY_OPTIONS ('overwriteSchema' = 'true'): overwriteSchema is not the COPY INTO target-evolution setting and would not express the requested additive merge. mergeSchema is the supported copy option for evolving the target schema.

COPY_OPTIONS ('force' = 'true'): force disables idempotent file tracking so previously loaded files are reprocessed. It affects which files are loaded, not whether new columns are added.

FORMAT_OPTIONS ('inferSchema' = 'true'): Reader-side inference concerns parsing source data; it does not merge the resulting schema into the target table. Target evolution is separately controlled by the copy option.

References:

1. **COPY INTO**: <https://docs.databricks.com/aws/en/sql/language-manual/delta-copy-into>

Question 48

A `report_failure` task depends on three validation tasks. It must run only when the validations did not all pass, and it should be excluded when all three succeed.

Which Run if condition should `report_failure` use?

- A. All failed, which then runs the task after the validations have finished if each of the three validations has failed during that job run
- B. At least one failed, which then runs the task once the validations have finished if any one of the three has failed during that job run
- C. None failed, which then runs the task once the validations have finished and at least one of the three has actually run during that job run
- D. All done, which then runs the task after the validations have finished, whatever the individual results of the three are during that job run

Correct answer: B

B. At least one failed, which then runs the task once the validations have finished if any one of the three has failed during that job run

Explanation

Correct answer:

At least one failed, which then runs the task once the validations have finished if any one of the three has failed during that job run

Why this is correct:

“At least one failed” runs after its dependencies finish when any validation task reports failure. If all three succeed, none failed, so the failure-reporting task is excluded; if any one fails, the condition matches.

Why the other options are wrong:

All failed, which then runs the task after the validations have finished if each of the three validations has failed during that job run: “All failed” is narrower than requested: it requires every validation to fail. A single failed check among otherwise successful checks would not trigger the report.

None failed, which then runs the task once the validations have finished and at least one of the three has actually run during that job run: “None failed” has the opposite purpose and is satisfied when validations succeed without failure. It would run the notification task in the successful case.

All done, which then runs the task after the validations have finished, whatever the individual results of the three are during that job run: “All done” runs regardless of outcomes, including when every validation passes. The task must depend on failure state, not only completion.

References:

1. Job | Databricks API Reference: <https://docs.databricks.com/api/jobs/v2/job>

Question 49

The eligible external Delta table `corp.raw.stock` must become a Unity Catalog managed table while keeping its existing name, grants, and table history intact.

Which approach meets this requirement?

- A. Run `CREATE OR REPLACE TABLE corp.raw.stock AS SELECT` from itself, which rewrites the data files into managed storage right in place
- B. Deep clone it over to `corp.raw.stock_managed`, drop the original table, and then rename the new clone back to the original table name
- C. Run `ALTER TABLE corp.raw.stock UNSET LOCATION`, which detaches the storage path so that Unity Catalog manages the existing files
- D. Run `ALTER TABLE corp.raw.stock SET MANAGED`, which moves its data files into managed storage while the table keeps its name and its identity

Correct answer: D

D. Run `ALTER TABLE corp.raw.stock SET MANAGED`, which moves its data files into managed storage while the table keeps its name and its identity

Explanation

Correct answer:

Run `ALTER TABLE corp.raw.stock SET MANAGED`, which moves its data files into managed storage while the table keeps its name and its identity

Why this is correct:

`ALTER TABLE ... SET MANAGED` converts an eligible external table in place. Unity Catalog moves the files into managed storage while retaining the table's identity and name, which is why this operation fits the requirement to keep the existing object and its history and grants.

Why the other options are wrong:

Run `CREATE OR REPLACE TABLE corp.raw.stock AS SELECT` from itself, which rewrites the data files into managed storage right in place: Replacing a table with a query rewrites or replaces table contents and is not the supported metadata-preserving conversion operation. It risks changing identity and history.

Run `ALTER TABLE corp.raw.stock UNSET LOCATION`, which detaches the storage path so that Unity Catalog manages the existing files: An external table cannot be made managed by merely removing its location. Conversion must move the files and update the table's storage management through `SET MANAGED`.

Deep clone it over to `corp.raw.stock_managed`, drop the original table, and then rename the new

clone back to the original table name: Dropping the existing table and substituting a clone introduces a new object lifecycle, risking the name's identity, grants, and history rather than converting the existing object in place.

References:

1. ALTER TABLE:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-alter-table>

Question 50

A bronze STRING column payload holds JSON such as {"sensor":"S4","value":17.5}. Silver needs a STRUCT with sensor STRING and value DOUBLE, and a malformed payload should yield NULL fields rather than fail the query.

- A. `schema_of_json(payload)`
- B. `get_json_object(payload, '$.sensor')`
- C. `from_json(payload, 'sensor STRING, value DOUBLE')`
- D. `parse_json(payload)`

Correct answer: C

C. `from_json(payload, 'sensor STRING, value DOUBLE')`

Explanation

Correct answer

`from_json(payload, 'sensor STRING, value DOUBLE')`

Why this is correct:

`from_json` parses JSON text against the supplied struct schema and returns a typed struct with sensor STRING and value DOUBLE. In its default PERMISSIVE mode, a malformed JSON string produces a struct whose fields are NULL instead of an error, which satisfies both the requested shape and the error handling.

Why the other options are wrong:

`parse_json(payload)` - `parse_json` returns a flexible VARIANT, not the explicitly typed STRUCT required by silver. It also does not apply the supplied field schema.

`schema_of_json(payload)` - `schema_of_json` reports a schema representation; it does not parse each payload into a row value. The field types must be declared to produce the requested struct.

`get_json_object(payload, '$.sensor')` - `get_json_object` extracts one path as a string, so it cannot return both requested fields in a typed struct. It is useful for selecting individual JSON properties.

References

- [from_json function](#)

Question 51

VACUUM ran on a Delta table yesterday with the default seven-day retention threshold. Today a query uses VERSION AS OF to read a version whose data files were replaced by an update twelve days ago.

What happens to the query, and why?

- A. It returns the current version instead, because time travel falls back to the latest snapshot when an older version's files are missing from storage
- B. It succeeds, because the transaction log still records that version, and Delta rebuilds the removed data files from its commit entries
- C. It fails, because VACUUM deleted every data file that no current version references once that file was older than the retention threshold
- D. It succeeds, because VACUUM only removes data files older than 30 days, which is the default retention for the table's own transaction log entries

Correct answer: C

C. It fails, because VACUUM deleted every data file that no current version references once that file was older than the retention threshold

Explanation

Correct answer:

It fails, because VACUUM deleted every data file that no current version references once that file was older than the retention threshold

Why this is correct:

Delta time travel needs both the transaction log and the historical data files referenced by the selected version. The log can still describe the version, but the 12-day-old files were eligible for removal when VACUUM applied the default seven-day deleted-file retention; metadata cannot reconstruct their contents.

Why the other options are wrong:

It succeeds, because the transaction log still records that version, and Delta rebuilds the removed data files from its commit entries: Transaction-log commits identify which files belong to a version, but they do not contain copies of the rows. Once VACUUM removes a required data file, there is nothing in the log from which to rebuild it.

It succeeds, because VACUUM only removes data files older than 30 days, which is the default retention for the table's own transaction log entries: The 30-day default applies to log history, not the files VACUUM removes. Time travel requires both forms of retention, so keeping the log alone is insufficient.

It returns the current version instead, because time travel falls back to the latest snapshot when an older version's files are missing from storage: Time travel does not silently substitute a different snapshot. Returning current data would produce a different answer than the requested historical version.

References:

1. **Work with table history:** <https://docs.databricks.com/aws/en/tables/history>

Question 52

A query writes 2,000 shuffle partitions, but most of them hold well under 1 MB, and task scheduling overhead dominates the stage's time. AQE is enabled on the cluster.

Which setting should be checked first before the stage is remeasured?

- A. A higher `spark.sql.shuffle.partitions` value, giving each shuffle partition less data to process and each of its tasks a shorter runtime
- B. `spark.sql.adaptive.coalescePartitions.enabled`, so AQE merges the small shuffle partitions into fewer, adequately sized partitions at runtime
- C. `spark.sql.autoBroadcastJoinThreshold` set to -1, stopping the Spark planner from choosing the broadcast joins that produce these small partitions
- D. `spark.sql.adaptive.skewJoin.enabled`, so AQE splits up the largest of the shuffle partitions into several smaller tasks during the join

Correct answer: B

B. `spark.sql.adaptive.coalescePartitions.enabled`, so AQE merges the small shuffle partitions into fewer, adequately sized partitions at runtime

Explanation

Correct answer:

`spark.sql.adaptive.coalescePartitions.enabled`, so AQE merges the small shuffle partitions into fewer, adequately sized partitions at runtime

Why this is correct:

AQE can coalesce small shuffle partitions after observing their runtime sizes. With 2,000 tiny partitions and scheduler overhead dominating, checking adaptive partition coalescing targets the excessive number of small tasks while preserving the query's logical result.

Why the other options are wrong:

A higher `spark.sql.shuffle.partitions` value, giving each shuffle partition less data to process and each of its tasks a shorter runtime: Increasing the initial partition count creates even more, smaller tasks, worsening scheduler overhead. It is appropriate when partitions are too large and parallelism is insufficient.

`spark.sql.autoBroadcastJoinThreshold` set to -1, stopping the Spark planner from choosing the broadcast joins that produce these small partitions: Broadcast thresholds influence join planning; they do not tell AQE to merge small shuffle partitions. Disabling broadcast could change the join strategy without addressing the observed task count.

`spark.sql.adaptive.skewJoin.enabled`, so AQE splits up the largest of the shuffle partitions into

several smaller tasks during the join: Skew optimization splits oversized partitions in supported shuffle joins. The issue here is that most partitions are tiny, so they need coalescing rather than splitting.

References:

1. Adaptive query execution: <https://docs.databricks.com/aws/en/optimizations/age>

Question 53

An engineer compares the output of `df.describe()` and `df.summary()` on a numeric price column. Both methods are called with their default arguments.

Which TWO statements are correct?

- A. `summary()` also returns the approximate 25%, 50%, and 75% percentile values for price in its default output
- B. Both of them return pandas DataFrames whose rows are collected onto the driver node before they are shown
- C. `describe()` also returns the approximate 25%, 50%, and 75% percentile values for price in its default output
- D. Both of them return count, mean, stddev, min, and max values for the price column in their default output
- E. `summary()` returns a query plan string for the price column instead of a actual DataFrame of statistics

Correct answers: A, D

- A. `summary()` also returns the approximate 25%, 50%, and 75% percentile values for price in its default output
- D. Both of them return count, mean, stddev, min, and max values for the price column in their default output

Explanation

Correct answer:

Both of them return count, mean, stddev, min, and max values for the price column in their default output

`summary()` also returns the approximate 25%, 50%, and 75% percentile values for price in its default output

Why this is correct:

Both methods compute count, mean, standard deviation, minimum, and maximum by default. `summary()` extends that basic descriptive output with approximate quartiles; `describe()` does not include those percentiles unless explicitly requested as statistics.

Why the other options are wrong:

Both of them return pandas DataFrames whose rows are collected onto the driver node before they are shown: PySpark returns Spark DataFrames, not pandas DataFrames collected to the driver. The result can be displayed or explicitly converted, but these methods themselves return distributed DataFrame objects.

summary() returns a query plan string for the price column instead of a actual DataFrame of statistics: summary() computes requested or default statistics and returns a DataFrame of results. It is not a query-plan inspection method.

describe() also returns the approximate 25%, 50%, and 75% percentile values for price in its default output: The five basic statistics are describe()'s default set. To request percentiles, use summary() or specify statistics explicitly rather than assuming they appear in describe().

References:

1. **pyspark.sql.DataFrame.describe:** <https://spark.apache.org/docs/4.0.3/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.describe.html>
2. **pyspark.sql.DataFrame.summary:** <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.summary.html>

Question 54

A job loads a nightly extract from an on-premises Oracle database that is not registered in Unity Catalog and writes no files to cloud storage. The extract usually finishes by 01:30.

Which trigger should the load job use?

- A. A file arrival trigger on the Oracle server's export directory, which the job can then watch through the JDBC connection details in its own settings
- B. A scheduled trigger set at 02:00, because nothing in Unity Catalog or cloud storage changes that an event-based trigger could watch for
- C. A continuous trigger, which keeps polling Oracle for changes and starts the load as soon as the nightly extract there has completed
- D. A table update trigger on a foreign table that federates the Oracle table into Unity Catalog, which then fires when Oracle's source data changes

Correct answer: B

B. A scheduled trigger set at 02:00, because nothing in Unity Catalog or cloud storage changes that an event-based trigger could watch for

Explanation

Correct answer:

A scheduled trigger set at 02:00, because nothing in Unity Catalog or cloud storage changes that an event-based trigger could watch for

Why this is correct:

A scheduled trigger is appropriate when the source is an external Oracle database with no Unity Catalog table event and no cloud file arrival to monitor. Starting at 02:00 gives the usual extract time to complete by 01:30 before the job begins its load.

Why the other options are wrong:

A file arrival trigger on the Oracle server's export directory, which the job can then watch through the JDBC connection details in its own settings: A file-arrival trigger monitors a Unity Catalog volume or external location; it cannot watch a server directory through JDBC connection settings. No file transfer to cloud storage occurs here.

A continuous trigger, which keeps polling Oracle for changes and starts the load as soon as the nightly extract there has completed: Continuous mode starts another job run when the previous run ends; it is not a database polling trigger and does not wait for Oracle's nightly extract to complete.

A table update trigger on a foreign table that federates the Oracle table into Unity Catalog, which then fires when Oracle's source data changes: The source is not registered in Unity Catalog, so there

is no foreign-table update event to monitor. Even federation would expose source data for querying rather than make this batch load start at a chosen time.

References:

1. Automate jobs with schedules and triggers: <https://docs.databricks.com/aws/en/jobs/triggers>

Question 55

Marketing data lives in a SaaS tool that has no Lakeflow Connect managed connector. The team wants a supported, low-code way to land it in Unity Catalog tables without writing API code.

Which approach fits?

- A. An ingestion partner that is connected through Partner Connect, which provisions a SQL warehouse and a service principal for its loads
- B. Lakehouse Federation over the SaaS tool, which lands the tool's objects in Unity Catalog tables each time that a user queries them
- C. Auto Loader pointed directly at the SaaS tool's API endpoint, which discovers new records the same way that it discovers new files in storage
- D. A Lakeflow Connect managed pipeline built anyway, choosing the SaaS tool's name from the generic connector list offered during setup

Correct answer: A

A. An ingestion partner that is connected through Partner Connect, which provisions a SQL warehouse and a service principal for its loads

Explanation

Correct answer:

An ingestion partner that is connected through Partner Connect, which provisions a SQL warehouse and a service principal for its loads

Why this is correct:

Partner Connect provides an integration route to supported ingestion partners when Databricks has no managed Lakeflow Connect connector for the SaaS product. The partner's low-code integration can land data into Unity Catalog without requiring the team to implement the vendor API itself.

Why the other options are wrong:

Lakehouse Federation over the SaaS tool, which lands the tool's objects in Unity Catalog tables each time that a user queries them: Federation exposes supported external systems for querying in place; it does not copy arbitrary SaaS records into managed Unity Catalog tables on query.

Auto Loader pointed directly at the SaaS tool's API endpoint, which discovers new records the same way that it discovers new files in storage: Auto Loader discovers files from cloud storage paths, not records from a SaaS API endpoint. An API integration needs a connector or ingestion partner to retrieve records.

A Lakeflow Connect managed pipeline built anyway, choosing the SaaS tool's name from the generic connector list offered during setup: Lakeflow Connect offers source-specific managed

connectors rather than a universal generic connector for any SaaS vendor. If the source is absent from supported connectors, use an integration partner.

References:

1. Connect to ingestion partners using Partner Connect:

<https://docs.databricks.com/aws/en/partner-connect/ingestion>